

# Flow Logic for Process Calculi

HANNE RIIS NIELSON, FLEMMING NIELSON, and HENRIK PILEGAARD,

The Technical University of Denmark

3

Flow Logic is an approach to statically determining the behavior of programs and processes. It borrows methods and techniques from Abstract Interpretation, Data Flow Analysis and Constraint Based Analysis while presenting the analysis in a style more reminiscent of Type Systems. Traditionally developed for programming languages, this article provides a tutorial development of the approach of Flow Logic for process calculi based on a decade of research.

We first develop a simple analysis for the  $\pi$ -calculus; this consists of the specification, semantic soundness (in the form of subject reduction and adequacy results), and a Moore Family result showing that a least solution always exists, as well as providing insights on how to implement the analysis. We then show how to strengthen the analysis technology by introducing reachability components, interaction points, and localized environments, and finally, we extend it to a relational analysis.

A Flow Logic is a program logic—in the same sense that a Hoare's logic is. We conclude with an executive summary presenting the highlights of the approach from this perspective including a discussion of theoretical properties as well as implementation considerations.

The electronic supplements present an application of the analysis techniques to a version of the  $\pi$ -calculus incorporating distribution and code mobility; also the proofs of the main results can be found in the electronic supplements.

Categories and Subject Descriptors: D.2.4 [Software Engineering]: Software/Program Verification—*Formal methods*; F.3.1 [Logics and Meanings of Programs]: Specifying and Verifying and Reasoning about Programs—*Assertions, invariants, Logics of programs, Mechanical verification, Specification techniques*; F.3.2 [Logics and Meanings of Programs]: Semantics of Programming Languages—*Operational semantics, Process models, Program analysis*; F.4.1 [Mathematical Logic and Formal Languages]: Mathematical Logic—*Logic and constraint programming, Mechanical theorem proving*; I.2.2 [Artificial Intelligence]: Automatic Programming—*Program verification*

General Terms: Algorithms, Design, Documentation, Languages, Reliability, Theory, Verification

Additional Key Words and Phrases: Static analysis, flow logic, process calculi, Moore family, subject reduction, adequacy

## ACM Reference Format:

Nielson, H. R., Nielson, F., and Pilegaard, H. 2012. Flow logic for process calculi. *ACM Comput. Surv.* 44, 1, Article 3 (January 2012), 39 pages.

DOI = 10.1145/2071389.2071392 <http://doi.acm.org/10.1145/2071389.2071392>

## 1. INTRODUCTION

Given a process in a process calculus, or a program in some programming language, one might be interested in knowing certain properties that will hold in all executions.

---

Authors' address: H. R. Nielson, F. Nielson, and H. Pilegaard, Department of Informatics and Mathematical Modelling, The Technical University of Denmark, Richard Petersens Plads, Building 321, DK-2800 Kongens Lyngby, Denmark; email: {riis, nielson, hepi}@imm.dtu.dk.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from the Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701, USA, fax +1 (212) 869-0481, or [permissions@acm.org](mailto:permissions@acm.org).

© 2012 ACM 0360-0300/2012/01-ART3 \$10.00

DOI 10.1145/2071389.2071392 <http://doi.acm.org/10.1145/2071389.2071392>

Examples include knowing where values originate from, what are the possible values that expressions might evaluate to, what are the combination of values possible at a given program point.

Static analysis encompasses a number of approaches aimed at providing safe approximations to the dynamic behavior of the processes or programs of interest. In Nielson et al. [1999], we cover four of the main ones: Data Flow Analysis, Constraint Based Analysis, Abstract Interpretation, and Type and Effect Systems. All of these approaches were originally developed for programming languages and then subsequently some of these have also been developed for process calculi.

Flow Logic bridges the gap between these approaches and is a robust approach able to deal with a wide variety of programming languages and calculi of computation including calculi with functional, imperative, object-oriented, concurrent, distributed, and mobile features; we refer to the survey paper Nielson and Nielson [2002] for an overview of Flow Logic for programming languages.

In short, Flow Logic uses the methods and techniques prevalent in Data Flow Analysis, Constraint Based Analysis, and Abstract Interpretation, while orchestrating the development using the mindset from Type Systems. Indeed, Flow Logic makes a clear distinction between (1) the specification of the analysis, (2) whether or not a proposed analysis result is indeed sound with respect to the semantics, and (3) the computation of the best analysis result. It is a program logic in much the same sense that a Hoare's logic is; but it does not rely on a clear separation between pre- and post-condition. The logical format used for presenting specifications focuses on ensuring the implementability of the analyses—often in low polynomial (cubic) time.

Approaches based on type and effect systems, including session types, have a long history of dealing with process calculi, whereas the development of Flow Logic for process calculi has happened during the last decade and is the subject of the present article. We introduce and highlight the basic concept of Flow Logic for process calculi as developed in a decade of work and also provide concrete examples embodying these concepts.

## 1.1 Overview

We introduce the basic approach of Flow Logic in Section 2. Here we consider the polyadic  $\pi$ -calculus [Milner 1999] and develop a simple analysis tracking whether there might be arity mismatches in communications. We illustrate a number of applications of the analysis and also develop the relevant semantic correctness results: a subject reduction result and a number of adequacy results. We next prove the Moore Family result, which ensures that there is no non-determinism in our analysis, i.e., that there always is a least analysis result and we develop an algorithm for efficiently finding it. We conclude with a few pointers to the literature; the novice to Flow Logic might want to stop here on a first reading.

In Section 3 we demonstrate a number of analysis techniques for improving the precision of the analysis. In Section 3.1 we add a reachability component where we make sure not to analyze unreachable code, e.g. parts of processes following a communication that has no matching counterpart. Next in Section 3.2 we show how a careful analysis of the potential interaction points can improve the precision of the analysis result; basically this is the technique we use to differentiate between the parallel composition and the nondeterministic choice of processes. Then in Section 3.3 we show how to make use of localized environments that are able to differentiate between different binding occurrences of the same name. The analyses presented in this article generally use a rather straightforward approach to name bindings, where their relationship is not taken into account, but in Section 3.4 we develop a more complex analysis able to take

relations between name bindings into account. We conclude with a few pointers to the literature.

We raise the level of abstraction in Section 4 where we give a general characterization of what a Flow Logic for a process calculus is. This section is not intended for the novice to Flow Logic and is not required for straightforward adaptations of Flow Logic to other calculi. It highlights the main points that have been developed in the previous sections and provides firm guidance on how to adapt Flow Logic specifications to more demanding scenarios without jeopardizing the theoretical and practical properties that are the hallmark of Flow Logic.

We provide our concluding remarks in Section 5.

The article is supplemented with two electronic appendices. Appendix A shows the ease with which the Flow Logic approach is able to deal with new linguistic primitives; this takes the form of developing a Flow Logic along the lines of Section 2 for the  $D\pi$  calculus [Hennessy 2007]. Finally, Appendix B contains the proofs of the main results.

## 2. THE BASICS OF FLOW LOGIC

In this section we introduce the basic approach of Flow Logic for the polyadic  $\pi$ -calculus [Milner 1999]. We first introduce the process calculus and then develop the Flow Logic specification and go on to demonstrate the main properties of the specification. Semantic soundness is established by means of a subject reduction result and several adequacy results for several applications of the analysis. We give a theoretical treatment of how to obtain a solution by establishing a Moore Family result (as briefly mentioned in the Introduction) and then we give a more pragmatic treatment by outlining how a solver might be constructed. We conclude with a few pointers to the literature.

### 2.1 The Process Calculus

*Syntax.* We shall consider the following version of the  $\pi$ -calculus where the syntax of processes and actions is given by

$$\begin{aligned} P &::= (\text{new } n)P \mid P_1 \mid P_2 \mid !P \mid \sum_{i \in I} \pi_i.P_i \\ \pi &::= u(\vec{v}) \mid u(\vec{x}) \mid \tau. \end{aligned}$$

As in Milner [1999] processes  $P \in \text{Proc}$  can be constructed using process restriction  $(\text{new } n)P$ , parallel composition  $P_1 \mid P_2$ , replication  $!P$ , and guarded sums  $\sum_{i \in I} \pi_i.P_i$ . In its general form, the latter makes use of a finite index set  $I$ ; nullary sums are written  $\mathbf{0}$ , unary sums are written  $\pi.P$ , whereas binary sums are written  $\pi_1.P_1 + \pi_2.P_2$ . There are three different kinds of actions  $\pi \in \text{Act}$ , namely polyadic output  $u(\vec{v})$ , polyadic input  $u(\vec{x})$ , and silent actions  $\tau$ . In examples we often dispense with writing the continuation process when it is  $\mathbf{0}$ .

A central component of the  $\pi$ -calculus is that of names and we shall use  $u, v \in \text{Name}$  to range over names. A bound name  $u$  introduced by a restriction  $(\text{new } u)$  will be called a *constant* and we generally use  $n, m \in \text{Const}$  to denote this. Similarly, a bound name  $u$  introduced by an input  $v(\vec{u})$  will be called a *variable* and we generally use  $x, y \in \text{Var}$  to denote this. This distinction between constants and variables is common in programming languages and will be a useful guide when explaining the detailed operation of the Flow Logic; clearly it does not limit the expressivity of the process calculus.

In contrast to Milner [1999], we shall find it helpful for our presentation to formally partition the name space  $\text{Name}$  into the disjoint union of  $\text{Const}$  and  $\text{Var}$ , both of which provide infinite supplies of names that are constants, respectively variables, and to

Table I. Structural Congruence:  $P \equiv Q$ 

|                                                        |                                                                                                   |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Abelian monoid laws for parallel:                      | Scope laws:                                                                                       |
| $(P \mid Q) \mid R \equiv P \mid (Q \mid R)$           | $(\text{new } n)(\text{new } m)P \equiv (\text{new } m)(\text{new } n)P$                          |
| $P \mid Q \equiv Q \mid P$                             | $(\text{new } n)\mathbf{0} \equiv \mathbf{0}$                                                     |
| $P \mid \mathbf{0} \equiv P$                           | $(\text{new } n)(P \mid Q) \equiv (\text{new } n)P \mid Q \quad \text{if } n \notin \text{fc}(Q)$ |
| Unfolding of replication:                              | $\alpha$ -renaming:                                                                               |
| $!P \equiv P \mid !P$                                  | $P \equiv_\alpha Q \Rightarrow P \equiv Q$                                                        |
| Summands can be permuted in $\sum_{i \in I} \pi_i.P_i$ |                                                                                                   |

Table II. Reduction Relation:  $P \rightarrow P'$ 

|                                                                                                                                         |                                                                                                   |                                                                              |  |                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|--|-----------------------------------------------|
| $[\text{COM}](n(\vec{m}).P + P') \mid (n(\vec{x}).Q + Q') \rightarrow P \mid Q[\vec{m}/\vec{x}] \quad \text{if }  \vec{m}  =  \vec{x} $ |                                                                                                   |                                                                              |  | $[\text{TAU}] \quad \tau.P + Q \rightarrow P$ |
| $[\text{RES}] \quad \frac{P \rightarrow P'}{(\text{new } n)P \rightarrow (\text{new } n)P'}$                                            | $[\text{EQ}] \quad \frac{P \equiv Q \quad Q \rightarrow Q' \quad Q' \equiv P'}{P \rightarrow P'}$ | $[\text{PAR}] \quad \frac{P \rightarrow P'}{P \mid Q \rightarrow P' \mid Q}$ |  |                                               |

incorporate this into the syntax as shown in the preceding to be pedantic,  $\text{Name} = \text{Const} \cup \text{Var}$  and  $\text{Const} \cap \text{Var} = \emptyset$ . We use vector notation for sequences of such entities and write  $|\vec{u}|$  for the length of the sequence  $\vec{u}$ . Finally,  $\text{fc}(P)$  and  $\text{fv}(P)$  denote the free constants, respectively free variables, of  $P$ ; the definitions are straightforward and therefore omitted. We shall write  $P_\star$  for the main process of interest.

*Semantics.* We shall follow the approach of Milner [1999] and define the semantics by a structural congruence relation  $\equiv$  and a reduction relation  $\rightarrow$ . The *structural congruence* is defined by the laws in Table I together with the classical laws for an equivalence relation and the laws for replacement in context. The definition makes use of a notion of  $\alpha$ -renaming for names

$$(\text{new } n)P \equiv_\alpha (\text{new } m)P[m/n] \quad \text{if } m \notin \text{fc}(P).$$

Here  $P[u/v]$  is a process that is as  $P$  except that all free occurrences of the name  $v$  have been replaced by the name  $u$  while avoiding name capture. We shall later replace this notion of  $\alpha$ -renaming with a notion of *disciplined*  $\alpha$ -renaming that, without loss of generality, disciplines the choice of fresh names.

The *reduction relation* is specified in Table II. The axiom for communication checks whether the arities of the input and output match, and it makes use of the substitution operation  $P[\vec{u}/\vec{v}]$ , which is the generalization of the operation  $P[u/v]$  explained in the preceding, obtained by taking  $P[\vec{u}u/\vec{v}v] = (P[u/v])[\vec{u}/\vec{v}]$  for nonempty sequences. As usual we have rules for reduction in the context of restriction and parallel composition and we have a rule incorporating structural congruence. We refer to Milner [1999] for additional explanations.

*Example 2.1.* Consider the following system  $P_\star$  consisting of three parallel processes exchanging messages using a global channel  $c$

$$(\text{new } n)(c\langle n \rangle. c\langle n \rangle) \mid c(x).((\text{new } m)x\langle m \rangle) \mid c(y).y(x).$$

Let us assume that the two leftmost processes communicate in the first reduction step; then we get

$$(\text{new } n)(c\langle n \rangle \mid (\text{new } m)n\langle m \rangle) \mid c(y).y(x),$$

reflecting not only that  $n$  has been substituted for  $x$  in the second process but also that the scope of  $n$  has been extended to include both processes. The leftmost and the

rightmost processes communicate in the second reduction step, thereby substituting  $n$  for  $y$  in the rightmost process

$$(\text{new } n)(\mathbf{0} \mid (\text{new } m)n(m) \mid n(x)).$$

Finally, the two rightmost processes can communicate over the channel  $n$  and we are done.

A variant of the process called  $P'_\star$  has a slightly different rightmost process

$$(\text{new } n)(c(n).c(n)) \mid c(x).((\text{new } m)x(m)) \mid c(x, y).x(y).$$

After the initial communication over  $c$  it becomes

$$(\text{new } n)(c(n) \mid (\text{new } m)n(m)) \mid c(x, y).x(y),$$

but now the process is stuck: the communication over  $c$  is not possible since the arities of the output and input do not match.

Our choice of semantics restricts our attention to closed processes, that is, processes with no free names and hence processes that cannot interact with an unknown environment—thus the main process  $P_\star$  of interest will satisfy  $\text{fv}(P_\star) = \text{fc}(P_\star) = \emptyset$ . This will simplify the presentation in the rest of the article but it is in no way inherent to our approach; indeed in previous work (e.g. Bodei et al. [2001a, 1998, 1999]) we have performed similar developments based on the late semantics of Parrow [2001], thereby catering for open processes as well.

## 2.2 The Flow Logic Specification

*Abstract Domains.* The overall aim of our analysis is to determine whether the process  $P_\star$  of interest might enter a stuck configuration because of a mismatch between the arities in a communication step. In order to do so, the analysis will capture which sequences of constants might be communicated over the various channels—and in order to do that it, will also need to track the bindings of constants to variables. This motivates introducing the following abstract domains.

- $\rho : \text{Var} \rightarrow \wp(\text{Const})$  is the *abstract environment* that maps a variable to the set of constants that it might be bound to.
- $\kappa : \text{Const} \rightarrow \wp(\text{Const}^*)$  is the *abstract channel environment* that maps a (channel) constant to the set of sequences of constants that may be communicated over it.
- $\psi : \wp(\text{Const})$  is the *error component* that records the set of (channel) constants where there may be an arity mismatch in a communication.

These domains can be turned into complete lattices by extending the subset ordering  $\subseteq$  on  $\wp(\text{Const})$  in a pointwise manner. For ease of presentation we extend the abstract environment to  $\rho : (\text{Const} \cup \text{Var}) \rightarrow \wp(\text{Const})$  by setting  $\rho(n) = \{n\}$  for all constants  $n$ ; we prefer this to simply setting  $\rho : \text{Name} \rightarrow \wp(\text{Name})$ , as this would allow having, e.g.,  $\rho(m) = \{x\}$  for a name  $m$  and variable  $x$ , which is semantically impossible. Also we extend  $\rho$  to sequences  $\vec{u} = (u_1, \dots, u_k)$  of constants and variables in a pointwise manner so  $\rho(\vec{u}) = \rho(u_1) \times \dots \times \rho(u_k) = \{(m_1, \dots, m_k) \mid \bigwedge_{i=1}^k m_i \in \rho(u_i)\}$ .

*Judgements.* In Flow Logic the analysis is specified by logical judgements and in our analysis of the  $\pi$ -calculus they take the form

$$\rho, \kappa \vdash_P P : \psi.$$

This judgement expresses that  $\rho, \kappa$  and  $\psi$  provide a valid analysis result for the behavior of  $P$ —note that the judgement does not say that it is the best such analysis result nor does it give any guidelines for how to compute the analysis result itself.

Table III. Flow Logic:  $\rho, \kappa \vdash_P P : \psi$  and  $\rho, \kappa \vdash_A \pi : \psi$ 

|       |                                                                                                                                                                                                                                          |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [RES] | $\rho, \kappa \vdash_P (\text{new } n)P : \psi \text{ iff } \rho, \kappa \vdash_P P : \psi$                                                                                                                                              |
| [PAR] | $\rho, \kappa \vdash_P P_1 \mid P_2 : \psi \text{ iff } \rho, \kappa \vdash_P P_1 : \psi_1 \wedge \rho, \kappa \vdash_P P_2 : \psi_2 \wedge \psi_1 \cup \psi_2 \subseteq \psi$                                                           |
| [REP] | $\rho, \kappa \vdash_P !P : \psi \text{ iff } \rho, \kappa \vdash_P P : \psi$                                                                                                                                                            |
| [SUM] | $\rho, \kappa \vdash_P \sum_{i \in I} \pi_i.P_i : \psi \text{ iff } \forall i \in I : (\rho, \kappa \vdash_A \pi_i : \psi'_i \wedge \psi'_i \subseteq \psi \wedge \rho, \kappa \vdash_P P_i : \psi_i \wedge \psi_i \subseteq \psi)$      |
| [OUT] | $\rho, \kappa \vdash_A u(\vec{v}) : \psi \text{ iff } \forall n \in \rho(u) : \rho(\vec{v}) \subseteq \kappa(n)$                                                                                                                         |
| [IN]  | $\rho, \kappa \vdash_A u(\vec{x}) : \psi \text{ iff } \forall n \in \rho(u) : \kappa(n) \cap \text{Const}^{ \vec{x} } \subseteq \rho(\vec{x}) \wedge \kappa(n) \setminus \text{Const}^{ \vec{x} } \neq \emptyset \Rightarrow n \in \psi$ |
| [TAU] | $\rho, \kappa \vdash_A \tau : \psi \text{ iff true}$                                                                                                                                                                                     |

The idea is that  $\rho$  and  $\kappa$  contain global information that captures information about the behavior of the overall process  $P$ , of interest, whereas  $\psi$  captures local information of interest primarily for the process  $P$ . It is important to notice that the analysis result captures information about  $P$  as well as the potential processes it may evolve into.

The judgement is defined by a number of clauses: the first four clauses of Table III (to be explained in the following). We have exactly one clause for each of the syntactic forms of processes in the  $\pi$ -calculus. In the case of guarded sums we shall make use of a judgement defined for actions. It happens to have the same form as that for processes

$$\rho, \kappa \vdash_A \pi : \psi,$$

and expresses that  $\rho, \kappa$ , and  $\psi$  together form a valid analysis result for the action  $\pi$ . Again we have exactly one clause for each of the syntactic forms of actions: the last three clauses of Table III (to be explained in the following).

*Clauses.* We shall now explain the form of the individual clauses of Table III. We begin with the clause [RES] for  $(\text{new } n)P$ ; it expresses that  $\rho, \kappa$ , and  $\psi$  constitute a valid analysis result for  $(\text{new } n)P$  if and only if they do so for  $P$  alone; thus the clause (and hence the analysis) is oblivious to the scoping of constants.

The clause [PAR] for parallel composition states that  $\rho, \kappa$ , and  $\psi$  constitute a valid analysis result for  $P_1 \mid P_2$  whenever we can find valid analysis judgements for  $P_1$  and  $P_2$  using the same global components  $\rho$  and  $\kappa$  but possibly different local components  $\psi_1$  and  $\psi_2$ ; however, it must be the case that  $\psi_1 \cup \psi_2 \subseteq \psi$  reflecting that arity mismatches recorded for  $P_1$  and  $P_2$  also should be recorded for  $P_1 \mid P_2$ .

Turning to the clause [REP] for replication we immediately observe that it is oblivious to the replication operator. For simple analyses, like checking for potential mismatches of arities in communications, this is quite sufficient.

The fourth clause [SUM] is for the guarded sum and let us first take a look at unary sums where the clause specializes to

$$\rho, \kappa \vdash_P \pi.P : \psi \text{ iff } \rho, \kappa \vdash_A \pi : \psi' \wedge \psi' \subseteq \psi \wedge \rho, \kappa \vdash_P P : \psi'' \wedge \psi'' \subseteq \psi.$$

This expresses that  $\rho$  and  $\kappa$  must be valid analysis results for the action  $\pi$  as well as its continuation  $P$  and that the corresponding error components must be included in the error component associated with  $\pi.P$ . The clause for  $\sum_{i \in I} \pi_i.P_i$  of Table III generalizes this to hold for each of the summands  $\pi_i.P_i$ .

The remaining clauses of Table III define the judgement for actions, and it is in these clauses that we impose conditions on the information contained in  $\rho, \kappa$ , and  $\psi$ .



Let us consider the clause [OUT] for the output action  $u(\vec{v})$ . In the case where  $u$  as well as  $\vec{v}$  are constants it amounts to

$$\rho, \kappa \vdash_A n(\vec{m}) : \psi \quad \text{iff} \quad \vec{m} \in \kappa(n),$$

and this simply mimics that  $\vec{m}$  might be communicated over the channel  $n$  and hence this must be recorded in  $\kappa$ . Now  $u$  as well as  $\vec{v}$  could contain variables so the right-hand side of the clause has to be generalized, and here we use  $\rho$ , to capture all the possibilities as expressed by  $\forall n \in \rho(u) : \rho(\vec{v}) \subseteq \kappa(n)$ . Note that the clause does not impose any conditions on the error component  $\psi$ , as the output action on its own cannot reveal any arity mismatch in communication.

To explain the clause [IN] for the input action  $u(\vec{x})$  let us first consider the case where  $u$  is a constant; here the clause specializes to

$$\begin{aligned} \rho, \kappa \vdash_A n(\vec{x}) : \psi \quad \text{iff} \quad & \kappa(n) \cap \text{Const}^{|\vec{x}|} \subseteq \rho(\vec{x}) \wedge \\ & \kappa(n) \setminus \text{Const}^{|\vec{x}|} \neq \emptyset \Rightarrow n \in \psi. \end{aligned}$$

The set  $\kappa(n)$  contains all the sequences of constants  $\vec{m}$  that might be output to  $n$  somewhere in the process, and from the point of view of the analysis, any of them could be input over  $n$  at this particular point in the process. Provided that  $\vec{m}$  and  $\vec{x}$  have the same length, the  $i$ th constant of  $\vec{m}$  will be a potential value of the  $i$ th variable of  $\vec{x}$ ; this is expressed by the condition

$$\kappa(n) \cap \text{Const}^{|\vec{x}|} \subseteq \rho(\vec{x}),$$

and assuming that  $\vec{x} = (x_1, \dots, x_k)$  this amounts to

$$\forall (m_1, \dots, m_k) \in \kappa(n) \cap \text{Const}^k : m_1 \in \rho(x_1) \wedge \dots \wedge m_k \in \rho(x_k).$$

However,  $\kappa(n)$  might also contain sequences with lengths different from  $|\vec{x}|$  and then the communication might fail and we shall then require that  $n \in \psi$ ; this is captured by the condition  $\kappa(n) \setminus \text{Const}^{|\vec{x}|} \neq \emptyset \Rightarrow n \in \psi$ . The clause of Table III generalizes these requirements to the case where  $u \in \text{Const} \cup \text{Var}$ .

*Example 2.2.* Let us return to the process  $P_\star$  of Example 2.1. Taking  $\rho, \kappa$ , and  $\psi$  as follows

$$\rho = \begin{array}{|c|c|} \hline x & y \\ \hline \{n, m\} & \{n\} \\ \hline \end{array} \quad \kappa = \begin{array}{|c|c|c|} \hline c & n & m \\ \hline \{n\} & \{m\} & \{m\} \\ \hline \end{array} \quad \psi = \emptyset,$$

we obtain an acceptable analysis result as can easily be verified from the clauses of Table III. This is the most precise analysis result for  $P_\star$  and we can see that it correctly captures that  $x$  may be bound to  $n$  as well as  $m$ . However, it fails to notice that  $m$  will not be used for communication and obviously this is because the two defining occurrences of  $x$  are mixed up. It is easy to check that we can obtain a more precise analysis result if we avoid this and replace  $y(x)$  by, say,  $y(z)$  in the rightmost process.

For the process  $P'_\star$  of Example 2.1 we have the following acceptable analysis result

$$\rho = \begin{array}{|c|c|} \hline x & y \\ \hline \{n\} & \{m\} \\ \hline \end{array} \quad \kappa = \begin{array}{|c|c|c|} \hline c & n & m \\ \hline \{n\} & \{m\} & \emptyset \\ \hline \end{array} \quad \psi = \{c\},$$

reflecting that the process will indeed become stuck when trying to communicate over  $c$ .

It is characteristic for Flow Logic that there is exactly one clause for each syntactic construct. Thus we would never introduce a general subtyping clause stating for

example, that it is always possible to enlarge the error component of an analysis result. Such results, however, can often be established as formal consequences of the specification:

*Fact 2.3 Monotonicity.* If  $\rho, \kappa \vdash_P P : \psi$  and  $\psi \subseteq \psi'$  then  $\rho, \kappa \vdash_P P : \psi'$ .

*Well-definedness.* It is easy to argue that the analysis specified by Table III is well-defined as it is syntax-directed: the validity of an analysis result for a composite construct always relies on the validity of analysis results for syntactically smaller constructs.

### 2.3 The Subject Reduction Result

Semantic correctness amounts to ensuring that the judgement  $\rho, \kappa \vdash_P P : \psi$  correctly captures the behavior of the process  $P$ . We shall express this as a *subject reduction result* meaning that if we have an analysis result for  $P$ , and  $P$  evolves into some process  $Q$ , then the very same analysis result is also valid for  $Q$ .

Now if  $P$  evolves into  $Q$  then it also evolves into any process  $Q'$  that is structurally congruent to  $Q$ . This means that an analysis result that is valid for  $P$  also must be valid for  $Q$  and all processes that are obtained from  $Q$  by  $\alpha$ -renaming. However, the analysis results  $\rho, \kappa$ , and  $\psi$  are closely tailored to the constants and variables occurring in  $P$  and as a consequence it is impossible to establish the required connection.

The solution to this is to introduce equivalence classes of constants and make use of *disciplined  $\alpha$ -renaming* as expressed by

$$(\text{new } n)P \equiv_{\alpha} (\text{new } m)P[m/n] \quad \text{if } m \notin \text{fc}(P) \wedge [n] = [m].$$

Here  $[n]$  is the equivalence class corresponding to  $n$ ; it is also called the *canonical name* of  $n$  and is similar to the *sorts* of Milner [1999]. Clearly, we assume that there is an infinite supply of names within each equivalence class. Without loss of generality, we shall in the following, replace the law for  $\alpha$ -renaming in Table I with the preceding rule for disciplined  $\alpha$ -renaming.

The clauses specifying the analysis judgements refer to the constants of the initial process  $P_*$  of interest. We shall therefore take  $[n] = n$  for all constants  $n$  occurring in  $P_*$ . By taking  $[x] = x$  for all variables  $x$ , we can lift the operation  $[\cdot]$  to actions and processes in a pointwise manner. Clearly, for the initial process we will then have  $P_* = [P_*]$ , whereas this does not necessary hold for the derivatives of  $P_*$ .

We can now establish the following auxiliary results.

**LEMMA 2.4 STRUCTURAL CONGRUENCE.** *If  $P \equiv Q$  then  $\rho, \kappa \vdash_P [P] : \psi$  if and only if  $\rho, \kappa \vdash_P [Q] : \psi$ .*

**LEMMA 2.5 SUBSTITUTION.** *If  $\rho, \kappa \vdash_P [P] : \psi$  then  $\rho, \kappa \vdash_P [P[m/y]] : \psi$  provided that  $[m] \in \rho(y)$ .*

We are now ready to formalize and prove the subject reduction result.

**THEOREM 2.6 SUBJECT REDUCTION.** *If  $P \rightarrow Q$  and  $\rho, \kappa \vdash_P [P] : \psi$  then  $\rho, \kappa \vdash_P [Q] : \psi$ .*

### 2.4 Adequacy Results

We shall now give a few applications of the preceding analysis and present the associated adequacy results. We already noticed that there are processes that are stuck because the arities of the actions in a communication do not match and we now want to exclude them from consideration.



*Well-behaved processes.* To formalize this we shall first define a notion of *dynamic well-behavedness* based on the semantics and then a notion of *static well-behavedness* based on the Flow Logic. Our adequacy result will then show that static well-behavedness implies dynamic well-behavedness.

To express dynamic well-behavedness we shall first introduce the standard concept of a context; a *context*  $C$  is formally given by

$$C ::= (\text{new } n)C \mid C \mid P \mid [\cdot],$$

and essentially is a process with a hole  $[\cdot]$  where another process can be inserted; we write  $C[P]$  for the process obtained by inserting  $P$  in the hole of  $C$ . (Unlike substitution there is no  $\alpha$ -renaming to avoid name capture when plugging the hole in a context.)

We shall say that the process  $P_\star$  is dynamically well-behaved if whenever  $P_\star$  evolves into a process that is structurally congruent to one of the form  $C[(n\langle\vec{m}\rangle.R + R') \mid (n(\vec{x}).Q + Q')]$  for some context  $C$  then indeed  $|\vec{m}| = |\vec{x}|$  (meaning that the communication step will be possible).

We shall say that the process  $P_\star$  is statically well-behaved if there exist  $\rho$  and  $\kappa$  such that  $\rho, \kappa \vdash_P P_\star : \emptyset$ ; that is, there is a valid analysis result for  $P_\star$  where the error component  $\psi$  is empty. We now have the following.

**THEOREM 2.7 ADEQUACY FOR WELL-BEHAVED PROCESSES.** *If the process  $P_\star$  is statically well-behaved then it is also dynamically well-behaved.*

*Well-sorted processes.* We may go one step further and introduce a notion of well-sorted processes as suggested in Milner [1999]. Also in this case our analysis gives the necessary information for statically imposing well-sortedness on processes. Our approach is similar to the preceding: first we define a notion of dynamic well-sortedness based on the semantics, then we define a notion of static well-sortedness based on the analysis and finally we prove an adequacy result expressing that the static notion of well-sortedness implies the dynamic one.

Following Milner we shall first introduce a set of sorts and require that each constant  $n$  has a sort  $\sigma(n) \in \text{Sort}$ . The sorts are preserved by disciplined  $\alpha$ -renaming so if  $[n] = [m]$  then also  $\sigma(n) = \sigma(m)$ . We extend  $\sigma$  in a component-wise manner to work on tuples as well as sets of tuples. A *sorting* is then defined as a function  $\Sigma : \text{Sort} \rightarrow \text{Sort}^*$  that for each sort specifies a sequence of sorts describing not only the arity of the constants with that sort but also the sorts of the constants that may be communicated over it.

We shall now define that  $P_\star$  is *dynamically well-sorted* if whenever  $P_\star$  evolves into a process that is structurally congruent to one of the form  $C[(n\langle\vec{m}\rangle.R + R') \mid (n(\vec{x}).Q + Q')]$  then  $|\vec{m}| = |\vec{x}|$  as well as  $\Sigma(\sigma(n)) = \sigma(\vec{m})$ .

We shall say that the process  $P_\star$  is *statically well-sorted* if there exist  $\rho$  and  $\kappa$  such that  $\rho, \kappa \vdash_P P_\star : \emptyset$  and furthermore  $\sigma(\kappa(n)) \subseteq \{\Sigma(\sigma(n))\}$  for all constants  $n$ . We then have the following.

**THEOREM 2.8 ADEQUACY FOR WELL-SORTED PROCESSES.** *If the process  $P_\star$  is statically well-sorted then it is also dynamically well-sorted.*

*Non-leaking processes.* As a final example we shall assign security levels to the channels and use the analysis to impose a confidentiality policy on the processes, ensuring that they do not send constants with a high security level on channels with a lower security level; this is along the line of the development of Bodei et al. [1998, 1999, 2001a]. For simplicity we shall only impose two levels, low and high, and we want to ensure that only low information is communicated on low channels.

The idea is to introduce a mapping  $\Lambda : \text{Const} \rightarrow \text{Level}$  that to each constant associates a security level; we shall require that  $\lfloor n \rfloor = \lfloor m \rfloor$  ensures that  $\Lambda(n) = \Lambda(m)$  so that the security level will be preserved by disciplined  $\alpha$ -renaming. Let us write  $\text{Level} = \{\text{low}, \text{high}\}$  and define an ordering  $\sqsubseteq$  by  $\text{low} \sqsubseteq \text{high}$ .

Following this approach we shall say that the process  $P_\star$  is *dynamically non-leaking* if whenever  $P_\star$  evolves into a process that is structurally congruent to one of the form  $C[(n(\vec{m}).R + R') \mid (n(\vec{x}).Q + Q')]$  then  $|\vec{m}| = |\vec{x}|$  as well as  $\Lambda(\vec{m}) \sqsubseteq \Lambda(n)^{|\vec{m}|}$  – in the case where  $\Lambda(n) = \text{low}$  and this ensures that all the constants of  $\vec{m}$  will be of low security level and when  $\Lambda(n) = \text{high}$  it does not impose any restrictions.

We shall say that the process  $P_\star$  is *statically non-leaking* if there exist  $\rho$  and  $\kappa$  such that  $\rho, \kappa \vdash_P P_\star : \emptyset$  and furthermore  $\kappa$  satisfies  $\forall \vec{m} \in \kappa(n) : \Lambda(\vec{m}) \sqsubseteq \Lambda(n)^{|\vec{m}|}$  for all constants  $n$ . We then have the following.

**THEOREM 2.9 ADEQUACY FOR NON-LEAKING PROCESSES.** *If the process  $P_\star$  is statically non-leaking then it is also dynamically non-leaking.*

## 2.5 The Moore Family Result

The theorems presented so far have been concerned with interpreting the meaning of a valid analysis result for a process  $P$  in relation to its semantics and some properties of interest. However, can we be sure that  $P$  has any valid analysis results at all? And if it has more than one valid analysis result, is there a best one? These questions will be answered in the affirmative by the Moore Family result, also known as the *model intersection* property.

Formally, a *Moore Family* is a subset  $\mathcal{Y}$  of a complete lattice that is closed under greatest lower bounds, that is, it satisfies  $\forall Y \subseteq \mathcal{Y} : \sqcap Y \in \mathcal{Y}$ . We have the following

**PROPOSITION 2.10 MOORE FAMILY.** *The set  $\{(\rho, \kappa, \psi) \mid \rho, \kappa \vdash_P P : \psi\}$  is a Moore Family for all processes  $P$ .*

This result has the interesting corollary that all processes have a least, or best, analysis result. This follows by observing that the set

$$\{(\rho, \kappa, \psi) \mid \rho, \kappa \vdash_P P : \psi\}$$

is indeed a subset of the Moore Family and therefore  $\sqcap\{(\rho, \kappa, \psi) \mid \rho, \kappa \vdash_P P : \psi\}$  will be a valid analysis result for  $P$  and clearly it is the least such analysis result for  $P$ .

## 2.6 Implementation

The Moore Family result shows the existence of a least analysis result. To obtain an algorithm for constructing it we merely need to change our viewpoint.

Given a process  $P$  and the Flow Logic specification in Table III, we can unfold the defining clauses in Table III according to the syntax of  $P$  to obtain a formula  $\Phi$  such that the judgement  $\rho, \kappa \vdash_P P : \psi$  is equivalent to  $\Phi$ . As a simple example, in the case where  $P$  is  $c(n).\mathbf{0} \mid c(x).\mathbf{0}$  the formula  $\Phi$  becomes

$$\begin{aligned} & (\forall c' \in \rho(c) : \rho(n) \subseteq \kappa(c')) \quad \wedge \\ & (\forall c' \in \rho(c) : \kappa(c') \cap \text{Const} \subseteq \rho(x)) \quad \wedge \\ & (\forall c' \in \rho(c) : \kappa(c') \setminus \text{Const} \neq \emptyset \Rightarrow c' \in \psi), \end{aligned}$$

which can be further simplified to

$$\begin{aligned} & (\{n\} \subseteq \kappa(c)) \quad \wedge \\ & (\kappa(c) \cap \text{Const} \subseteq \rho(x)) \quad \wedge \\ & (\kappa(c) \setminus \text{Const} \neq \emptyset \Rightarrow c \in \psi), \end{aligned}$$

Table IV. Unfolding of the Flow Logic Specification

|          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| INPUT :  | a Flow Logic specification $\mathcal{F}$ (e.g. the one in Table III) and a process $P_*$                                                                                                                                                                                                                                                                                                                                                                        |
| OUTPUT : | a logical formula $\Phi_*$ such that $(\rho, \kappa, \psi) \models \Phi_*$ iff $\rho, \kappa \vdash_P P_* : \psi$                                                                                                                                                                                                                                                                                                                                               |
| METHOD : | initialise $\Phi_*$ to be $\rho, \kappa \vdash_P P_* : \psi$<br>while $\Phi_*$ contains an occurrence of $\tilde{R} \vdash_{P'} P' : \tilde{T}$<br>and there is a clause in $\mathcal{F}$ of the form $\alpha \text{ iff } \beta$<br>and a substitution $\theta$<br>such that $\theta\alpha$ equals the occurrence $\tilde{R} \vdash_{P'} P' : \tilde{T}$<br>do replace the occurrence of $\tilde{R} \vdash_{P'} P' : \tilde{T}$ with $\theta\beta$ in $\Phi_*$ |

using our extension of  $\rho$  to constants, e.g.  $\rho(c) = \{c\}$ . Next we can solve the constraints generated by computing the least values of the predicates used in  $\Phi$ , such that  $\Phi$  holds. Continuing the example, we need to compute the least values of  $\kappa(x)$  and  $\psi$  that satisfy the constraints; clearly this gives  $\kappa(x) = \{n\}$  and  $\psi = \emptyset$ .

Taking a somewhat more general approach, the acceptability judgement

$$\rho, \kappa \vdash_P P : \psi \quad \text{iff} \quad \Phi$$

associates each process  $P$  with a logical formula  $\Phi$ , such that an analysis estimate  $(\rho, \kappa, \psi)$  is acceptable for  $P$  if and only if  $(\rho, \kappa, \psi)$  constitutes a model of  $\Phi$ . This suggests a two-step algorithm. First we unfold the Flow Logic specification for a given process  $P_*$  into a formula  $\Phi_*$  of an appropriate formal system. Next we compute the least model  $(\rho, \kappa, \psi)$  of  $\Phi_*$  using an appropriate solver for the formal system. We now sketch the development of this two-step algorithm.

*Unfolding the Flow Logic specification.* Algorithmically we perform the unfolding by the iterative algorithm outlined in Table IV. It takes as input a Flow Logic specification  $\mathcal{F}$  (e.g. the one in Table III) and produces the formula  $\Phi_*$  by repeated replacement of instances of left-hand-sides with instances of right-hand-sides; this is formalized in the algorithm by the explicit use of substitutions.

The algorithm can be made deterministic by fixing a strategy for finding candidate judgements for unfolding, e.g. choosing the first occurrence in a depth-first traversal of the logical formula, but the details will be of no importance for us because the resulting formula can be freely rearranged using, for example, the commutative and associative laws for conjunction. Termination of the algorithm, and the finiteness of the logical formula  $\Phi_*$ , follows directly from the finiteness of the syntactic representation of  $P_*$  whenever the Flow Logic is syntax-directed as is the case for the one in Table III. More complex techniques are necessary when this is not the case.

*Example 2.11.* Continuing Example 2.1, the algorithm of Table IV can be used to unfold the judgement  $\rho, \kappa \vdash_P P'_* : \psi$  into a conjunction of the following six judgements for actions and eight set inclusions.

$$\begin{array}{lll}
\rho, \kappa \vdash_A c(n) : \psi_{11} & \rho, \kappa \vdash_A c(x) : \psi_{211} & \rho, \kappa \vdash_A c(x, y) : \psi_{221} \\
\rho, \kappa \vdash_A c(n) : \psi_{12} & \rho, \kappa \vdash_A x(m) : \psi_{212} & \rho, \kappa \vdash_A x(y) : \psi_{222} \\
\psi_1 \cup \psi_2 \subseteq \psi & \psi_{11} \subseteq \psi_1 & \psi_{211} \subseteq \psi_{21} & \psi_{221} \subseteq \psi_{22} \\
\psi_{21} \cup \psi_{22} \subseteq \psi_2 & \psi_{12} \subseteq \psi_1 & \psi_{212} \subseteq \psi_{21} & \psi_{222} \subseteq \psi_{22}
\end{array}$$

The six judgements on actions can be further expanded using a combination of set inclusions and first order logic, while leaving the eight set inclusions unchanged.

$$\begin{aligned}
& \forall n' \in \rho(c) : \rho(n) \subseteq \kappa(n') \\
& \forall n' \in \rho(c) : \rho(n) \subseteq \kappa(n') \\
& \forall n' \in \rho(c) : \kappa(n') \cap \mathbf{Const} \subseteq \rho(x) \wedge (\kappa(n') \setminus \mathbf{Const} \neq \emptyset) \Rightarrow n' \in \psi_{211} \\
& \forall n' \in \rho(x) : \rho(m) \subseteq \kappa(n') \\
& \forall n' \in \rho(c) : \kappa(n') \cap \mathbf{Const}^2 \subseteq \rho(x, y) \wedge (\kappa(n') \setminus \mathbf{Const}^2 \neq \emptyset) \Rightarrow n' \in \psi_{221} \\
& \forall n' \in \rho(x) : \kappa(n') \cap \mathbf{Const} \subseteq \rho(y) \wedge (\kappa(n') \setminus \mathbf{Const} \neq \emptyset) \Rightarrow n' \in \psi_{222} \\
& \psi_1 \cup \psi_2 \subseteq \psi \quad \psi_{11} \subseteq \psi_1 \quad \psi_{211} \subseteq \psi_{21} \quad \psi_{221} \subseteq \psi_{22} \\
& \psi_{21} \cup \psi_{22} \subseteq \psi_2 \quad \psi_{12} \subseteq \psi_1 \quad \psi_{212} \subseteq \psi_{21} \quad \psi_{222} \subseteq \psi_{22}
\end{aligned}$$

The formula  $\Phi_*$  is the conjunction of these constraints.

*Solving the constraints.* Having obtained the logical formula  $\Phi_*$  the next problem is to find its least (or best) model  $(\rho, \kappa, \psi)$ . One obvious approach is to develop a special purpose constraint solver handling exactly the combination of set inclusions and first order logic that we have used; essentially this is the approach taken in Table 3.7 of Nielson et al. [1999]. An alternative, and more general approach, is to transform the constraints into an appropriate fragment of first order logic and then use an off-the-shelf solver. We shall illustrate this approach in the following example, where we recast the constraints within Datalog [Apt et al. 1988; Chandra and Harel 1980] and then use The Succinct Solver [Nielson et al. 2002b] for computing the least solution—we shall discuss this approach in more detail in Section 4.3.

*Example 2.12.* Returning to the constraints of Example 2.11, the idea is that each of the analysis components  $\rho$ ,  $\kappa$ , and  $\psi$  are turned into predicates and the constraints are then rephrased using first order logic. Let us first consider the eight subset inclusions. They take one of two forms, as exemplified by  $\psi_{11} \subseteq \psi_1$  and  $\psi_1 \cup \psi_2 \subseteq \psi$ , and they will be rewritten as

$$\forall n' : \psi_{11}(n') \Rightarrow \psi_1(n') \quad \text{and} \quad \forall n' : \psi_1(n') \vee \psi_2(n') \Rightarrow \psi(n').$$

Next consider the constraints generated for the three output actions. A typical case is  $\forall n' \in \rho(x) : \rho(m) \subseteq \kappa(n')$  that becomes

$$\forall n' : \rho(x, n') \Rightarrow (\forall m' : \rho(m, m') \Rightarrow \kappa(n', m'))$$

Finally consider the constraints generated for the three input actions. Here we exploit the fact that a simple inspection of the process  $P'_*$  shows that communication only occurs with arity one and two. First let us consider one of the unary input actions, namely  $\forall n' \in \rho(x) : (\kappa(n') \cap \mathbf{Const} \subseteq \rho(y)) \wedge ((\kappa(n') \setminus \mathbf{Const}) \neq \emptyset) \Rightarrow n' \in \psi_{222}$ . Here we rewrite the constraint as follows.

$$\begin{aligned}
& \forall n' : \rho(x, n') \Rightarrow (\forall m' : \kappa(n', m') \Rightarrow \rho(y, m')) \wedge \\
& \quad (\exists m'_1 : \exists m'_2 : \kappa(n', m'_1, m'_2)) \Rightarrow \psi_{222}(n').
\end{aligned}$$

For the binary input action we have  $\forall n' \in \rho(c) : \kappa(n') \cap \mathbf{Const}^2 \subseteq \rho(x, y) \wedge (\kappa(n') \setminus \mathbf{Const}^2 \neq \emptyset) \Rightarrow n' \in \psi_{221}$ , and we rewrite it as

$$\begin{aligned}
& \forall n' : \rho(c, n') \Rightarrow (\forall m'_1 : \forall m'_2 : \kappa(n', m'_1, m'_2) \Rightarrow \rho(x, m'_1) \wedge \rho(y, m'_2)) \wedge \\
& \quad (\exists m' : \kappa(n', m')) \Rightarrow \psi_{221}(n').
\end{aligned}$$

The conjunction of the formulae obtained in this way is in Datalog and the Succinct Solver [Nielson et al. 2002b] will produce the solution already presented in Example 2.2.

## 2.7 Bibliographical Notes

As discussed in the Introduction, the development of Flow Logic for process calculi involves transferring methods and techniques from Data Flow Analysis, Constraint Based Analysis, and Abstract Interpretation from the world of programming languages to the world of process calculi. The differences between programming languages and process calculi have presented a few obstacles that have not always been solved as elegantly as presented in the present article. We now give an overview of some of these issues.

*Names versus constants and variables.* The distinction between constants and variables is quite standard in programming languages but is absent in many process calculi. We do consider it useful when developing the static analysis although it is not formally necessary to do so. First of all, our presentation in Section 2.1 clearly shows that the distinction can always be made implicitly, based on the binding occurrence. Second, we would obtain exactly the same least solution regardless of whether we use  $\rho : (\text{Const} \cup \text{Var}) \rightarrow \wp(\text{Const})$  or  $\rho : \text{Name} \rightarrow \wp(\text{Name})$  as spurious content like  $\rho(m) = \{x\}$  for a name  $m$  and variable  $x$  would never arise in the least solution. However, maintaining a clear distinction serves as a useful typing discipline that facilitates a less error prone development of the Flow Logic specification—a lesson we learned the hard way in the early work of [Bodei et al. 1998, 1999, 2001a].

*Structural congruence and canonical names.* Starting with the  $\pi$ -calculus many process calculi are given a semantics that has two components. One is a structural congruence that allows rearranging processes in a way that should clearly be invariant under any semantic observations. The other is a reduction relation, whether in the form of a reaction semantics as used here or a labelled transition system, for modelling the actual computation steps. One can dispense with the structural congruence, at the price of a more complex specification of the reduction relation, and indeed this is the approach of early process calculi and of programming languages.

One of the complications offered by the structural congruence is the ability to perform  $\alpha$ -renaming of names—usually only for constants. Since the analyses generally track the way constants are propagated through the process as part of the computation steps, it is clear that names cannot be used directly in carrying analysis information. The solution presented here is the use of canonical names and disciplined  $\alpha$ -renaming and in such a way that we only analyze processes where all names are indeed canonical (as clearly displayed in Theorem 2.6).

The early work of Bodei et al. [1998, 1999, 2001a] takes a slightly different route. Here each defining occurrence is annotated with a *marker*  $\chi$  as in  $(\text{new } n^\chi)P$ . The markers are nothing but pointers into the process and thus remain stable under  $\alpha$ -renaming; hence they can be used to carry analysis information, but the presentation is syntactically heavier.

A similar approach is taken in the early work on analyzing the ambient calculus [Nielson et al. 2002]. The terminology is that markers for constants are called *stable names* whereas markers for variables are called *binders*. In later work Nielson et al. [2005] abandons the  $\alpha$ -renaming of variables and hence the binders, but then it introduces a notion of *groups* as a replacement for the stable names. Each defining occurrence of a constant is now associated with a group, as in  $(\text{new } n : \mu)P$ , and then an

additional construct  $(\text{new } \mu)P$  is used to introduce the group names. Essentially this means that the groups are *sorts* in the sense of Section 2.4.

The notion of canonical names was introduced for the LySa calculus in Bodei et al. [2005]; clearly the canonical names play a role similar to that of markers, groups, and stable names. However, the presentation in Bodei et al. [2005] lacks the idea of only analyzing processes where all names are indeed canonical (as in Theorem 2.6).

*Reaction semantics versus labelled transition systems.* As already mentioned, for process calculi we often have the choice between using a reaction semantics (as in this article) or using a labelled transition system [Pilegaard et al. 2006a]. Both approaches are equally amenable to the development of a Flow Logic and have been illustrated in many papers.

In our later papers we have generally favored the use of reaction semantics because we find that it gives a somewhat more concise definition of the subject reduction result and of the dynamic notions captured in the adequacy results. The reason quite simply is that accounting for the labels on transitions, some of which are input and others are output, sometimes lead to formulations that are more than three times more verbose than is the case for reaction semantics; this in turn gives rise to more complex proofs (as found in e.g. Bodei et al. [1998, 1999, 2001a]).

*Replication versus recursion.* There are at least two ways to equip process calculi with the ability to perform recursive invocations. One is the use of replication, as performed in this article, which seems to be the approach favored by many researchers and which is technically rather easy to deal with. Another is the use of explicit recursion operators, either in the form of named constants that are unfolded, or in the form of an explicit recursion construct embedded in the syntax. We refer to Nielson et al. [2004, 2007] and Tolstrup et al. [2007] for Flow Logics, taking this approach. In the case where the recursion constants take parameters, the development becomes more demanding.

### 3. EXTENDING THE ANALYSIS TECHNOLOGY

In Section 2 we have shown how to perform a full development of a Flow Logic for the  $\pi$ -calculus. In this section we illustrate a few techniques that can increase the precision for the examples considered in Section 2. The techniques are presented in order of increasing sophistication and are mainly orthogonal so that they can be combined in different ways; the analysis presented in Section 3.4 may be omitted on a first reading. For each of the extensions we shall establish a subject reduction result; when the proofs go beyond straightforward extensions of previous proofs we give the details in the Appendix. We shall refrain from going into the details about adequacy results and Moore Family results just as we omit discussions about how to implement the analyses—these parts of the development can be carried out much as in Section 2 and hence do not provide essential new insights. We conclude with a few pointers to the literature.

#### 3.1 Reachability

A process  $P$  is *dead* if it can never be executed. We might want to use our analysis to identify dead subprocesses in the process  $P_*$  of interest, and furthermore, we might want the analysis result of  $P_*$  not to be polluted by the analysis result of dead subprocesses. It turns out that it is very simple to modify the analysis of Section 2 to achieve this.

The idea is to extend the judgements of actions to contain yet another component

$$\rho, \kappa \vdash_A \pi : \psi \ \& \ \delta.$$



Table V. Flow Logic with a Reachability Component:  $\rho, \kappa \vdash_A \pi : \psi \ \& \ \delta$ 

|       |                                                                                                                                                                                                                                                                                                                                               |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [RES] | $\rho, \kappa \vdash_P (\text{new } n)P : \psi \iff \rho, \kappa \vdash_P P : \psi$                                                                                                                                                                                                                                                           |
| [PAR] | $\rho, \kappa \vdash_P P_1 \mid P_2 : \psi \iff \rho, \kappa \vdash_P P_1 : \psi_1 \wedge \rho, \kappa \vdash_P P_2 : \psi_2 \wedge \psi_1 \cup \psi_2 \subseteq \psi$                                                                                                                                                                        |
| [REP] | $\rho, \kappa \vdash_P !P : \psi \iff \rho, \kappa \vdash_P P : \psi$                                                                                                                                                                                                                                                                         |
| [SUM] | $\rho, \kappa \vdash_P \sum_{i \in I} \pi_i.P_i : \psi \iff \forall i \in I : (\rho, \kappa \vdash_A \pi_i : \psi'_i \ \& \ \delta_i \wedge \psi'_i \subseteq \psi \wedge \delta_i \neq \emptyset \Rightarrow (\rho, \kappa \vdash_P P_i : \psi_i \wedge \psi_i \subseteq \psi))$                                                             |
| [OUT] | $\rho, \kappa \vdash_A u(\vec{v}) : \psi \ \& \ \delta \iff \forall n \in \rho(u) : \rho(\vec{v}) \neq \emptyset \Rightarrow (\rho(\vec{v}) \subseteq \kappa(n) \wedge \bullet \in \delta)$                                                                                                                                                   |
| [IN]  | $\rho, \kappa \vdash_A u(\vec{x}) : \psi \ \& \ \delta \iff \forall n \in \rho(u) : (\kappa(n) \cap \text{Const}^{ \vec{x} } \neq \emptyset \Rightarrow \kappa(n) \cap \text{Const}^{ \vec{x} } \subseteq \rho(\vec{x}) \wedge \bullet \in \delta) \wedge \kappa(n) \setminus \text{Const}^{ \vec{x} } \neq \emptyset \Rightarrow n \in \psi$ |
| [TAU] | $\rho, \kappa \vdash_A \tau : \psi \ \& \ \delta \iff \bullet \in \delta$                                                                                                                                                                                                                                                                     |

Here  $\delta \subseteq \{\bullet\}$  is a *reachability component*: if  $\delta = \emptyset$  then it means that neither the action  $\pi$ , nor its continuation, can ever be executed. On the other hand, if  $\delta = \{\bullet\}$ , then both  $\pi$  and its continuation might be executed. This information can be used in the analysis of guarded sums; as an example consider the clause for unary sums

$$\rho, \kappa \vdash_P \pi.P : \psi \iff \rho, \kappa \vdash_A \pi : \psi' \ \& \ \delta \wedge \psi' \subseteq \psi \wedge \delta \neq \emptyset \Rightarrow (\rho, \kappa \vdash_P P : \psi'' \wedge \psi'' \subseteq \psi),$$

where the reachability component  $\delta$  from the analysis of the action  $\pi$  is used to determine whether or not the analysis result needs to be acceptable for the continuation  $P$ . This clause is generalized to guarded sums in Table V; the remaining clauses for processes are as in Table III.

The clauses defining the analysis of actions are also listed in Table V. In the clause [OUT] for output  $u(\vec{v})$ , we inspect whether the sequence  $\vec{v}$  indeed denotes anything; if it does not then the action can never be part of an interaction and we refrain from imposing that  $\bullet \in \delta$ . In the clause [IN] for input  $u(\vec{x})$ , we consult the abstract channel environment  $\kappa$  to determine whether there are any suitable sequences that might be communicated over the channel; if not then the action will never be part of an interaction and again we shall not impose that  $\bullet \in \delta$ . The silent action  $\tau$  is always possible so the clause [TAU] always imposes that  $\bullet \in \delta$ .

*Example 3.1.* The following values of  $\rho, \kappa$ , and  $\psi$  now constitute an acceptable analysis result for the process  $P'_*$  of Example 2.1

$$\rho = \begin{array}{|c|c|} \hline x & y \\ \hline \{n\} & \emptyset \\ \hline \end{array} \quad \kappa = \begin{array}{|c|c|c|} \hline c & n & m \\ \hline \{n\} & \{m\} & \emptyset \\ \hline \end{array} \quad \psi = \{c\}.$$

Compared to Example 2.2 we are now able to discover that neither the action  $c(x, y)$  nor its continuation  $x(y)$  will ever be executed and therefore we can have  $\rho(y) = \emptyset$ .

The correctness result of Theorem 2.6 carries over to the present setting.

**THEOREM 3.2 SUBJECT REDUCTION.** *If  $P \rightarrow Q$  and  $\rho, \kappa \vdash_P [P] : \psi$  then  $\rho, \kappa \vdash_P [Q] : \psi$ .*

Table VI. Sets of Interaction Points in  $P$ :  $\text{IP}(P)$ 

|                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{IP}((\text{new } n)P) = \text{IP}(P)$                                                                                                                                                                    |
| $\text{IP}(P_1 \mid P_2) = \text{IP}(P_1) \cup \text{IP}(P_2) \cup$<br>$(\text{Lab}_{\text{oa}}(P_1) \times \text{Lab}_{\text{ia}}(P_2)) \cup (\text{Lab}_{\text{oa}}(P_2) \times \text{Lab}_{\text{ia}}(P_1))$ |
| $\text{IP}(!P) = \text{Lab}_{\text{oa}}(P) \times \text{Lab}_{\text{ia}}(P)$                                                                                                                                    |
| $\text{IP}(\sum_{i \in I} \pi_i.P_i) = \bigcup_{i \in I} \text{IP}(P_i)$                                                                                                                                        |

### 3.2 Interaction Points

It is clear from the semantics that two actions can only interact if they are in different threads; this is not captured in the current analysis, where we happily record the result of a communication over  $c$  for a process like  $c\langle n \rangle.c(x)$ . We shall now show how to improve the analysis in this respect.

We extend the syntax to have labels  $\ell \in \text{Lab}$  on input and output actions

$$\pi ::= u\langle \vec{v} \rangle^\ell \mid u(\vec{x})^\ell \mid \tau.$$

These labels have no semantic meaning—they merely serve as pointers into the syntax. We shall write  $\text{Lab}_{\text{oa}}(P)$  for the set of labels of output actions in  $P$  and similarly  $\text{Lab}_{\text{ia}}(P)$  for the set of labels of input actions in  $P$ .

We can now determine the pairs of (labels of) output and input actions that might occur in parallel threads at some time during the execution of a process  $P$ . Such pairs  $(\ell_o, \ell_i)$  of labels will be called *interaction points*. The potential sets of interaction points  $\text{IP}(P) \subseteq \text{Lab}_{\text{oa}}(P) \times \text{Lab}_{\text{ia}}(P)$  of a process  $P$  can easily be determined by a simple syntax directed definition, as shown in Table VI. Note that for parallel composition we include all the possibilities for the two threads to interact, whereas this is not the case for sums. Replication  $!P$  corresponds to any number of parallel occurrences of  $P$  and hence all combinations of output and input labels from  $\text{Lab}_{\text{oa}}(P)$  and  $\text{Lab}_{\text{ia}}(P)$  are possible interaction points.

It is worth pointing out that  $\text{IP}(P)$  identifies a potential set of interaction points. For a process like  $c\langle n \rangle.c(x) \mid (c(y).c(y) + c(z).c(z))$ , it will predict that the output  $c\langle n \rangle$  may interact with any of the inputs of the two branches of the sum and similarly that the input  $c(x)$  may interact with any of the two outputs of the sum—however, in a concrete execution, one of the latter possibilities will be discarded depending on the first communication.

We can now refine our analysis to use the following abstract domains:

- $\rho : \text{Var} \rightarrow \wp(\text{Const})$  is as before: it maps a variable to the set of constants that it might be bound to.
- $\kappa : \text{Const} \times \text{Lab} \rightarrow \wp(\text{Const}^*)$  is extended to record which sequences of constants that might be communicated over a given (channel) constant at a given output label.
- $\psi : \wp(\text{Const} \times \text{Lab} \times \text{Lab})$  is extended to record the set of triples of (channel) constants and interaction points where there may be an arity mismatch in a communication.

The form of the judgements for processes as well as actions are as in Section 2 and so are the clauses for processes, that is, they are as in Table III. The clauses for actions are modified as shown in Table VII and now make use of the set of potential interaction points for the process  $P_*$  of interest. In the clause [OUT] for output  $u\langle \vec{v} \rangle^{\ell_o}$ , we simply record the relevant information for  $\ell_o$ . In the clause [IN] for the input action  $u(\vec{x})^{\ell_i}$ , we restrict our attention to the interaction points of  $\text{IP}(P_*)$  involving  $\ell_i$ , and furthermore, in the case of a potential mismatch of arity we require that the name of the channel as well as the interaction point be recorded in  $\psi$ .

Table VII. Flow Logic with Interaction Points:  $\rho, \kappa \vdash_A \pi : \psi$ 

|       |                                                                                                                                                                                                                                                                                                                                                                                |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [OUT] | $\rho, \kappa \vdash_A u(\vec{v})^{\ell_o} : \psi \quad \text{iff} \quad \forall n \in \rho(u) : \rho(\vec{v}) \subseteq \kappa(n, \ell_o)$                                                                                                                                                                                                                                    |
| [IN]  | $\rho, \kappa \vdash_A u(\vec{x})^{\ell_i} : \psi \quad \text{iff} \quad \forall n \in \rho(u) : \forall \ell_o : (\ell_o, \ell_i) \in \text{IP}(P_\star) \Rightarrow$<br>$(\kappa(n, \ell_o) \cap \text{Const}^{ \vec{x} } \subseteq \rho(\vec{x}) \wedge$<br>$\kappa(n, \ell_o) \setminus \text{Const}^{ \vec{x} } \neq \emptyset \Rightarrow (n, \ell_o, \ell_i) \in \psi)$ |
| [TAU] | $\rho, \kappa \vdash_A \tau : \psi \quad \text{iff} \quad \text{true}$                                                                                                                                                                                                                                                                                                         |

*Example 3.3.* Let us add labels to the process  $P'_\star$  considered in Example 2.1:

$$(\text{new } n)(c\langle n \rangle^1 . c\langle n \rangle^2) \mid c(x)^3 . ((\text{new } m)x\langle m \rangle^4) \mid c(x, y)^5 . x(y)^6$$

The set of output labels is  $\{1, 2, 4\}$ , the set of input labels is  $\{3, 5, 6\}$  and the set  $\text{IP}(P'_\star)$  of interaction points contains all combinations of output and input labels except for  $(4, 3)$ . The analysis result of Example 2.2 now becomes

$$\rho = \begin{array}{|c|c|} \hline x & y \\ \hline \{n\} & \{m\} \\ \hline \end{array} \quad \kappa = \begin{array}{c|c|c|c|} \ell_o & \kappa(c, \ell_o) & \kappa(n, \ell_o) & \kappa(m, \ell_o) \\ \hline 1 & \{n\} & \emptyset & \emptyset \\ 2 & \{n\} & \emptyset & \emptyset \\ 4 & \emptyset & \{m\} & \emptyset \\ \hline \end{array} \quad \psi = \left\{ \begin{array}{l} (c, 1, 5), \\ (c, 2, 5) \end{array} \right\}$$

thereby giving a more precise account of where an error might arise.

*Correctness.* It is important to note that the analysis of a process  $P$  is relative to the set  $\text{IP}(P_\star)$  of potential interaction points of the initial process—this is evident in the clause for [IN]. So to be precise we really should write the clauses as  $\rho, \kappa \vdash_P^{|\text{IP}(P_\star)|} P : \psi$  rather than  $\rho, \kappa \vdash_P P : \psi$ . Using this notation we can establish subject reduction as follows.

**THEOREM 3.4 SUBJECT REDUCTION.** *Assume that  $P_\star \rightarrow \dots \rightarrow P$ . If  $P \rightarrow Q$  and  $\rho, \kappa \vdash_P^{|\text{IP}(P_\star)|} [P] : \psi$  then  $\rho, \kappa \vdash_P^{|\text{IP}(P_\star)|} [Q] : \psi$ .*

The proof is similar to that of Theorem 2.6—the only complication arises for the communication axiom  $(n\langle \vec{m} \rangle^{\ell_o} . R + R') \mid (n(\vec{x})^{\ell_i} . Q + Q') \rightarrow P \mid Q[\vec{m}/\vec{x}]$  (where  $|\vec{m}| = |\vec{x}|$ ). Here we must additionally ensure that  $(\ell_o, \ell_i) \in \text{IP}(P_\star)$  as otherwise the Flow Logic would fail to capture the semantic behavior. For this the following lemma is helpful.

**LEMMA 3.5 INTERACTION POINTS.** *If  $P \equiv Q$  then  $\text{IP}(P) = \text{IP}(Q)$ . If  $P \rightarrow Q$  then  $\text{IP}(P) \supseteq \text{IP}(Q)$ .*

### 3.3 Localized Environments

So far we have used a global abstract environment  $\rho$  that is unable to differentiate between different binding occurrences of the same name. We shall now introduce localized environments as one solution to deal with this problem; this technique will prove useful in more complex analyses as will be illustrated later.

As in Section 3.2 we shall add labels  $\ell \in \text{Lab}$  to the actions; however, we only need the labels for the input actions, as they are the only ones that introduce new bindings to variables

$$\pi ::= u(\vec{v}) \mid u(\vec{x})^\ell \mid \tau.$$

We shall write  $\text{lab}(\pi, \ell)$  for the label of  $\pi$  in the case where  $\pi$  is an input action and otherwise it is simply  $\ell$ .

Table VIII. Flow Logic with Localized Environments:  $\rho, \kappa \vdash_P^\ell P : \psi$  and  $\rho, \kappa \vdash_A^{\ell, X} \pi : \psi$ 

|       |                                                                                                                                                                                                                                                                                                                                                                            |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [RES] | $\rho, \kappa \vdash_P^\ell (\text{new } n)P : \psi \text{ iff } \rho, \kappa \vdash_P^\ell P : \psi$                                                                                                                                                                                                                                                                      |
| [PAR] | $\rho, \kappa \vdash_P^\ell P_1 \mid P_2 : \psi \text{ iff } \rho, \kappa \vdash_P^\ell P_1 : \psi_1 \wedge \rho, \kappa \vdash_P^\ell P_2 : \psi_2 \wedge \psi_1 \cup \psi_2 \subseteq \psi$                                                                                                                                                                              |
| [REP] | $\rho, \kappa \vdash_P^\ell !P : \psi \text{ iff } \rho, \kappa \vdash_P^\ell P : \psi$                                                                                                                                                                                                                                                                                    |
| [SUM] | $\rho, \kappa \vdash_P^\ell \sum_{i \in I} \pi_i.P_i : \psi \text{ iff } \forall i \in I : (\rho, \kappa \vdash_A^{\ell, \text{fv}(P_i)} \pi_i : \psi'_i \wedge \psi'_i \subseteq \psi \wedge \rho, \kappa \vdash_P^{\text{lab}(\pi_i, \ell)} P_i : \psi_i \wedge \psi_i \subseteq \psi)$                                                                                  |
| [OUT] | $\rho, \kappa \vdash_A^{\ell, X} u\langle \vec{v} \rangle : \psi \text{ iff } \forall n \in \rho(\ell, u) : \rho(\ell, \vec{v}) \subseteq \kappa(n)$                                                                                                                                                                                                                       |
| [IN]  | $\rho, \kappa \vdash_A^{\ell, X} u(\vec{x})^{\ell_i} : \psi \text{ iff } \forall n \in \rho(\ell, u) : \kappa(n) \cap \text{Const}^{ \vec{x} } \subseteq \rho(\ell_i, \vec{x}) \wedge \forall y \in X \setminus \{\vec{x}\} : \rho(\ell, y) \subseteq \rho(\ell_i, y) \wedge \kappa(n) \setminus \text{Const}^{ \vec{x} } \neq \emptyset \Rightarrow (n, \ell_i) \in \psi$ |
| [TAU] | $\rho, \kappa \vdash_A^{\ell, X} \tau : \psi \text{ iff true}$                                                                                                                                                                                                                                                                                                             |

We shall modify the analysis domains to use labels as indicated here.

- $\rho : \text{Lab} \times \text{Var} \rightarrow \wp(\text{Const})$  is the *localized abstract environment*, that given a label, maps a variable to the set of constants that it can be bound to in the context described by the label.
- $\kappa : \text{Const} \rightarrow \wp(\text{Const}^*)$  is as before: it maps a (channel) constant to the set of sequences of constants that may be communicated over it.
- $\psi : \wp(\text{Const} \times \text{Lab})$  is the error component that now records pairs of (channel) constants and (input) labels indicating where an arity mismatch might occur.

The analysis of processes is now specified by judgements of the form

$$\rho, \kappa \vdash_P^\ell P : \psi,$$

where  $\ell$  identifies the context in which  $P$  is being analyzed; we shall assume that the initial process  $P_\star$  is analyzed in the context  $\ell_\star \in \text{Lab}$ . The analysis of actions is specified by judgements of the form

$$\rho, \kappa \vdash_A^{\ell, X} \pi : \psi.$$

Here  $X \subseteq \text{Var}$  is the set of variables occurring free in the continuation of  $\pi$ . The clauses are specified in Table VIII and are explained in the following.

The clauses for processes are much as before except that the context information is propagated to the subcomponents and the context information may only change in the case of guarded sums. We shall explain two instances of this clause in detail.

Let us first consider a unary sum with an output action; here the clause [SUM] specializes to

$$\rho, \kappa \vdash_P^\ell u\langle \vec{v} \rangle.P : \psi \text{ iff } \rho, \kappa \vdash_A^{\ell, \text{fv}(P)} u\langle \vec{v} \rangle : \psi' \wedge \psi' \subseteq \psi \wedge \rho, \kappa \vdash_P^\ell P : \psi'' \wedge \psi'' \subseteq \psi,$$

and we note that the continuation  $P$  is analyzed in the same context  $\ell$  as  $u\langle \vec{v} \rangle.P$ . The clause [OUT] for output ensures that the binding information is used correctly in the given context; in particular, the values of  $u$  as well as  $\vec{v}$  are obtained from the current context  $\ell$  as expressed by the requirement  $\forall n \in \rho(\ell, u) : \rho(\ell, \vec{v}) \subseteq \kappa(n)$  of the clause. Here  $\rho(\ell, (v_1, \dots, v_k))$  is shorthand for  $\rho(\ell, v_1) \times \dots \times \rho(\ell, v_k)$ , i.e.  $\{(m_1, \dots, m_k) \mid \bigwedge_{i=1}^k m_i \in \rho(\ell, v_i)\}$ .

Let us next consider a unary sum with an input action; then the clause [SUM] specializes to

$$\rho, \kappa \vdash_{\mathbf{P}}^{\ell} u(\vec{x})^{\ell_i}.P : \psi \text{ iff } \begin{array}{l} \rho, \kappa \vdash_{\mathbf{A}}^{\ell, \text{fv}(P)} u(\vec{x})^{\ell_i} : \psi' \wedge \psi' \subseteq \psi \wedge \\ \rho, \kappa \vdash_{\mathbf{P}}^{\ell_i} P : \psi'' \wedge \psi'' \subseteq \psi, \end{array}$$

and now we note that the continuation  $P$  is analyzed in the new context  $\ell_i$ , reflecting that new bindings are performed by the input action. The clause [IN] for input ensures that the relevant bindings of variables in  $\ell$  are propagated to the context  $\ell_i$ . For the bound variables of  $\vec{x}$  this is ensured by the requirement

$$\forall n \in \rho(\ell, u) : \kappa(n) \cap \text{Const}^{|\vec{x}|} \subseteq \rho(\ell_i, \vec{x}),$$

which is a shorthand for

$$\forall n \in \rho(\ell, u) : \forall (n_1, \dots, n_k) \in (\kappa(n) \cap \text{Const}^{|\vec{x}|}) : \bigwedge_{j=1}^k \{n_j\} \subseteq \rho(\ell_i, x_j),$$

where  $x_j$  is the  $j$ th element of  $\vec{x}$ . For the other variables of  $\text{fv}(P)$ , the propagation of the relevant bindings is ensured by the requirement

$$\forall y \in (\text{fv}(P) \setminus \{\vec{x}\}) : \rho(\ell, y) \subseteq \rho(\ell_i, y),$$

which expresses that these variables have the same potential values in the new and the old contexts. Finally, arity mismatches are recorded in  $\psi$ .

*Example 3.6.* Let us once again return to the process  $P_{\star}$  of Example 2.1 and add labels to the input actions as follows.

$$(\text{new } n)(c(n).c(n)) \mid c(x)^1.((\text{new } m)x(m)) \mid c(x, y)^2.x(y)^3.$$

Using localized environments, we obtain an acceptable analysis result when we take  $\rho, \kappa$  and  $\psi$  to be

|          |                |                 |                 |
|----------|----------------|-----------------|-----------------|
|          | $\ell$         | $\rho(\ell, x)$ | $\rho(\ell, y)$ |
| $\rho =$ | $\ell_{\star}$ | $\emptyset$     | $\emptyset$     |
| 1        |                | $\{n\}$         | $\emptyset$     |
| 2        |                | $\emptyset$     | $\emptyset$     |
| 3        |                | $\emptyset$     | $\emptyset$     |

|            |         |         |             |
|------------|---------|---------|-------------|
| $\kappa =$ | $c$     | $n$     | $m$         |
|            | $\{n\}$ | $\{m\}$ | $\emptyset$ |

$$\psi = \{(c, 2)\}.$$

This analysis is able to distinguish between the two defining occurrences of  $y$  and it also records that the input action labelled 3 will never be executed.

*Correctness.* As before the correctness of the analysis is expressed as a subject reduction result that requires a few auxiliary lemmas. The new substitution result reads as follows.

**LEMMA 3.7 SUBSTITUTION.** *If  $\rho, \kappa \vdash_{\mathbf{P}}^{\ell} [P] : \psi$  then  $\rho, \kappa \vdash_{\mathbf{P}}^{\ell} [P[m/y]] : \psi$  provided that  $[m] \in \rho(\ell, y)$ .*

The main complication in proving the subject reduction result arises for the communication axiom where we also need a relocation lemma.

**LEMMA 3.8 RELOCATION.** *If  $\rho, \kappa \vdash_{\mathbf{P}}^{\ell'} [P] : \psi$  then  $\rho, \kappa \vdash_{\mathbf{P}}^{\ell} [P] : \psi$  provided that  $\forall z \in \text{fv}(P) : \rho(\ell, z) \subseteq \rho(\ell', z)$ .*

We are now finally able to establish the subject reduction result.

**THEOREM 3.9 SUBJECT REDUCTION.** *If  $P \rightarrow Q$  and  $\rho, \kappa \vdash_P^\ell [P] : \psi$  then  $\rho, \kappa \vdash_P^\ell [Q] : \psi$ .*

### 3.4 Relational Analysis

The analyses presented so far are not able to record the relationship between the bindings of names. As an example consider the process

$$c\langle n, m \rangle \mid c\langle m, n \rangle \mid c\langle x, y \rangle.$$

The analyses will correctly record that both  $x$  and  $y$  take values from the set  $\{n, m\}$  but they fail to capture that  $x$  and  $y$  will be different. This gives rise to imprecision and we shall now see how to do better. Indeed, the analyses considered so far are *independent attribute* analyses [Nielson et al. 1999].

To improve on this aspect we shall now introduce a *relational analysis*. It will borrow ideas from the previous analyses but rather than recording the potential values of the variables by mappings  $\text{Var} \rightarrow \wp(\text{Const})$ , we shall fix the ordering of the variables in the process and record their potential values by sets of sequences of values of appropriate length. Each of these sequences corresponds to a potential environment where the  $i$ th variable has the  $i$ th value in the sequence. On top of this we shall localize the information much as in Section 3.3. Returning to the preceding motivating example, we may fix the ordering of the variables as  $(x, y)$  and then record their possible values by the set of pairs  $\{(n, m), (m, n)\}$ —the first components of these pairs record the possible values for  $x$  and the second components, the possible values for  $y$  and at the same time we have also recorded that  $x$  and  $y$  have different values.

The first step in our development is to add labels  $\ell \in \text{Lab}$  to the actions and here we follow the approach of Section 3.2 and add labels to all actions

$$\pi ::= u\langle \vec{v} \rangle^\ell \mid u\langle \vec{x} \rangle^\ell \mid \tau^\ell.$$

We shall write  $\text{lab}(\pi)$  for the label of  $\pi$  and we shall write  $\text{bv}(\pi)$  for the sequence of variables bound in  $\pi$  (writing  $\epsilon$  for the empty sequence).

We need to capture part of the control structure of the processes, and in particular, we need to fix the ordering of the variables at the various program points. To this effect let us consider the process  $P_\star$  of interest and let us assume that all the actions in  $P_\star$  are uniquely labelled. Furthermore we shall assume that  $P_\star$  will be analyzed in the context of the label  $\ell_\star$  not occurring in  $P_\star$ . We shall now introduce the following functions  $B$  and  $L$  (or  $B_\star$  and  $L_\star$  to be precise).

- $B.\text{Lab} \hookrightarrow \text{Lab}$  is a (partial) function that maps a given label to the label in which scope it occurs within  $P_\star$ .
- $L.\text{Lab} \hookrightarrow \text{Var}^*$  is a (partial) function that maps a given label to the sequence of variables in scope after performing the bindings of the labelled action in  $P_\star$ .

In the sequel we shall find it helpful to write  $B.\ell$  for  $B(\ell)$  and similarly  $L.\ell$  for  $L(\ell)$ .

*Example 3.10.* To illustrate the information captured by  $B$  and  $L$  let us consider the process  $P_\star$  that is a variant of the running example where the channel  $c$  is used to send pairs of names

$$((\text{new } n)(\text{new } m)c\langle n, m \rangle^1.c\langle m, n \rangle^2) \mid (c\langle x, y \rangle^3.x\langle y \rangle^4) \mid (c\langle x, y \rangle^5.y\langle y \rangle^6).$$

The auxiliary information recorded by  $B$  and  $L$  is as follows.

$$B = \begin{array}{|c|c|c|c|c|c|c|} \hline \ell_\star & 1 & 2 & 3 & 4 & 5 & 6 \\ \hline \ell_\star & \ell_\star & 1 & \ell_\star & 3 & \ell_\star & 5 \\ \hline \end{array} \quad L = \begin{array}{|c|c|c|c|c|c|c|} \hline \ell_\star & 1 & 2 & 3 & 4 & 5 & 6 \\ \hline \epsilon & \epsilon & \epsilon & xy & xy & xy & xyy \\ \hline \end{array}.$$



Table IX. Auxiliary Information for the Relational Analysis

|                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\mathcal{B}_\ell((\text{new } n)P) = \mathcal{B}_\ell(P)$                                                                                                                          |
| $\mathcal{B}_\ell(P_1 \mid P_2) = \mathcal{B}_\ell(P_1) \oplus \mathcal{B}_\ell(P_2)$                                                                                               |
| $\mathcal{B}_\ell(!P) = \mathcal{B}_\ell(P)$                                                                                                                                        |
| $\mathcal{B}_\ell(\Sigma_{i \in I} \pi_i.P_i) = \bigoplus_{i \in I} (\mathcal{B}_{\text{lab}(\pi_i)}(P_i) \oplus [\text{lab}(\pi_i) \mapsto \ell])$                                 |
| $\mathcal{L}_{\vec{y}}((\text{new } n)P) = \mathcal{L}_{\vec{y}}(P)$                                                                                                                |
| $\mathcal{L}_{\vec{y}}(P_1 \mid P_2) = \mathcal{L}_{\vec{y}}(P_1) \oplus \mathcal{L}_{\vec{y}}(P_2)$                                                                                |
| $\mathcal{L}_{\vec{y}}(!P) = \mathcal{L}_{\vec{y}}(P)$                                                                                                                              |
| $\mathcal{L}_{\vec{y}}(\Sigma_{i \in I} \pi_i.P_i) = \bigoplus_{i \in I} (\mathcal{L}_{\vec{y} \text{bv}(\pi_i)}(P_i) \oplus [\text{lab}(\pi_i) \mapsto \vec{y} \text{bv}(\pi_i)])$ |

The mapping  $\mathbf{B}$  determines the scope of each of the labels:  $\mathbf{B}.1$  is the scope of the action  $c\langle n, m \rangle^1$  and it is equal to  $\ell_*$ ,  $\mathbf{B}.2$  is the scope of the action  $c\langle m, n \rangle^2$ , and it is equal to 1, and so on.

The mapping  $\mathbf{L}$ , on the other hand, determines which variables are in scope after each of the labels. So we see that  $\mathbf{L}.1$  records the variables in scope after the action  $c\langle n, m \rangle^1$  has been performed and since there are none, the result is  $\epsilon$ . More interestingly,  $\mathbf{L}.5$  records that after the action labelled 5 has been executed, the variables  $x$  and  $y$  are in scope and they are ordered as  $xy$ . The action labelled 6 introduces a new binding occurrence of  $y$  and this is recorded in  $\mathbf{L}.6$  by the sequence  $xyy$  meaning that we will record the old as well as the new binding occurrences of the variable  $y$ .

To define the functions  $\mathbf{B}$  and  $\mathbf{L}$ , we shall use the two auxiliary functions  $\mathcal{B}_\ell$  (for  $\ell \in \text{Lab}$ ) and  $\mathcal{L}_{\vec{y}}$  (for  $\vec{y} \in \text{Var}^*$ ) defined in Table IX and explained in the following. The function  $\mathcal{B}_\ell$  will, given a process  $P$ , compute the scoping information for the labels of  $P$  under the assumption that the process itself is in the scope  $\ell$ . The definition is syntax directed and we use the function  $\oplus$  to merge mappings with disjoint domains—the assumption that  $P_*$  is uniquely labelled ensures that the mappings constructed for the subprocesses indeed have disjoint domains. The only nontrivial clause of the definition is for sums; in the case of prefixing it amounts to

$$\mathcal{B}_\ell(\pi.P) = \mathcal{B}_{\text{lab}(\pi)}(P) \oplus [\text{lab}(\pi) \mapsto \ell],$$

thereby recording that the scope of (the label of)  $\pi$  is  $\ell$  and that the continuation  $P$  is inspected in the scope of  $\text{lab}(\pi)$ .

The function  $\mathcal{L}_{\vec{y}}$  will, given a process  $P$ , determine the sequences of variables that are in scope at the various labels in  $P$  under the assumption that the context of  $P$  already tells us that the variables of  $\vec{y}$  are in scope. Also here, only the case of sums is nontrivial; in the case of prefixing it amounts to

$$\mathcal{L}_{\vec{y}}(\pi.P) = \mathcal{L}_{\vec{y} \text{bv}(\pi)}(P) \oplus [\text{lab}(\pi) \mapsto \vec{y} \text{bv}(\pi)].$$

Thus we inspect the continuation  $P$  in a context where the bound variables of  $\pi$  are appended to those of  $\vec{y}$  and we record that the same sequence of variables is of interest at the label  $\text{lab}(\pi)$ .

Finally we shall define the functions  $\mathbf{B}$  and  $\mathbf{L}$  for the process  $P_*$  of interest by the equations

$$\mathbf{B} = \mathcal{B}_{\ell_*}(P_*) \oplus [\ell_* \mapsto \ell_*]$$

$$\mathbf{L} = \mathcal{L}_\epsilon(P_*) \oplus [\ell_* \mapsto \epsilon],$$

where  $\ell_*$  is a label not occurring in  $P_*$ .

Table X. Relational Flow Logic:  $\rho, \kappa \vdash_P P : \psi$  and  $\rho, \kappa \vdash_A \pi : \psi$ 

|       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [RES] | $\rho, \kappa \vdash_P (\text{new } n)P : \psi \text{ iff } \rho, \kappa \vdash_P P : \psi$                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| [PAR] | $\rho, \kappa \vdash_P P_1 \mid P_2 : \psi \text{ iff } \rho, \kappa \vdash_P P_1 : \psi_1 \wedge \rho, \kappa \vdash_P P_2 : \psi_2 \wedge \psi_1 \cup \psi_2 \subseteq \psi$                                                                                                                                                                                                                                                                                                                                                                                 |
| [REP] | $\rho, \kappa \vdash_P !P : \psi \text{ iff } \rho, \kappa \vdash_P P : \psi$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| [SUM] | $\rho, \kappa \vdash_P \sum_{i \in I} \pi_i.P_i : \psi \text{ iff } \forall i \in I : (\rho, \kappa \vdash_A \pi_i : \psi'_i \wedge \psi'_i \subseteq \psi \wedge \rho, \kappa \vdash_P P_i : \psi_i \wedge \psi_i \subseteq \psi)$                                                                                                                                                                                                                                                                                                                            |
| [OUT] | $\rho, \kappa \vdash_A u(\vec{v})^\ell : \psi \text{ iff } \rho(\mathbf{B}.\ell) \subseteq \rho(\ell) \wedge \{\Pi_{u\vec{v}@\mathbf{L}(\mathbf{B}.\ell)}(\vec{m}) \mid \vec{m} \in \rho(\mathbf{B}.\ell)\} \subseteq \kappa$                                                                                                                                                                                                                                                                                                                                  |
| [IN]  | $\rho, \kappa \vdash_A u(\vec{x})^\ell : \psi \text{ iff } \left\{ \vec{m}\vec{o} \left  \begin{array}{l} \vec{m} \in \rho(\mathbf{B}.\ell) \wedge n = \Pi_{u@\mathbf{L}(\mathbf{B}.\ell)}(\vec{m}) \wedge \\ n\vec{o} \in \kappa \wedge  \vec{o}  =  \vec{x}  \end{array} \right. \right\} \subseteq \rho(\ell) \wedge \left\{ n \left  \begin{array}{l} \vec{m} \in \rho(\mathbf{B}.\ell) \wedge n = \Pi_{u@\mathbf{L}(\mathbf{B}.\ell)}(\vec{m}) \wedge \\ n\vec{o} \in \kappa \wedge  \vec{o}  \neq  \vec{x}  \end{array} \right. \right\} \subseteq \psi$ |
| [TAU] | $\rho, \kappa \vdash_A \tau^\ell : \psi \text{ iff } \rho(\mathbf{B}.\ell) \subseteq \rho(\ell)$                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

We are now ready to define the relational analysis. The judgements for processes and actions take the rather familiar form

$$\rho, \kappa \vdash_P P : \psi \quad \text{and} \quad \rho, \kappa \vdash_A \pi : \psi,$$

but some of the domains have been modified.

- $\rho.\text{Lab} \rightarrow \wp(\text{Const}^*)$  is the *abstract environment* that for each label contains sets of tuples of potential constants bound to the variables; we shall ensure that all tuples in  $\rho(\ell)$  have the same length as  $\mathbf{L}.\ell$ .
- $\kappa.\wp(\text{Const}^+)$  contains nonempty tuples, where the first component is the (channel) constant over which the remaining components might be communicated.
- $\psi.\wp(\text{Const})$  is as before, it records the set of (channel) constants where there may be an arity mismatch in a communication.

The clauses are specified in Table X. First we observe that the clauses [RES], [PAR], [REP], and [SUM] are much as in Table III.

Before discussing the nontrivial clauses for actions, we need to define the value  $\Pi_{u@\vec{z}}(\vec{m})$  of a name  $u$  with respect to a local environment  $\vec{m}$  specifying the values of the variables  $\vec{z}$  (and hence satisfying  $|\vec{m}| = |\vec{z}|$ ). Writing  $\vec{z} = (z_1, \dots, z_h)$  and  $\vec{m} = (m_1, \dots, m_h)$ , we proceed by cases on whether the name  $u$  is a constant  $n$  or a variable  $y$

$$\begin{aligned} \Pi_{n@\vec{z}}(\vec{m}) &= n \\ \Pi_{y@\vec{z}}(\vec{m}) &= m_i \text{ when } i \text{ is maximal such that } z_i = y. \end{aligned}$$

This means that if a variable  $y$  has several binding occurrences, that is, it occurs several times in  $\vec{z}$ , then  $\Pi_{y@\vec{z}}(\vec{m})$  returns the value in  $\vec{m}$  corresponding to the most recent (rightmost) binding of  $y$ . As an example we have  $\Pi_{y@xyy}(nmn) = n$ , reflecting that the value of  $y$  in the local environment  $nmn$  is  $n$ —given that the variables are ordered as  $xyy$  and that we are looking for the value of  $y$  in the most recent scope.

The clause [OUT] for an output action  $u(\vec{v})^\ell$  forwards the values in the existing environment into the new one. The new environment is given by the label  $\ell$  and we use the function  $\mathbf{B}$  to determine the existing environment, which is  $\mathbf{B}.\ell$  so we simply have

$\rho(\mathbf{B}.\ell) \subseteq \rho(\ell)$ . Furthermore, the clause records the values being output by considering each local environment  $\vec{m}$  of  $\rho(\mathbf{B}.\ell)$  in turn. Here the corresponding variables are listed in  $\mathbf{L}(\mathbf{B}.\ell)$  and the values corresponding to  $u$  and  $\bar{v}$  must be extracted and recorded in  $\kappa$ . To express this we make use of a slight generalization of the previous function

$$\Pi_{u_1 \dots u_k @ \vec{z}}(\vec{m}) = \Pi_{u_1 @ \vec{z}}(\vec{m}) \cdots \Pi_{u_k @ \vec{z}}(\vec{m}),$$

in order to collect the values in the right order; in the case of the [OUT] clause we require that  $\Pi_{u\bar{v} @ \mathbf{L}(\mathbf{B}.\ell)}(\vec{m}) \in \kappa$  reflecting that the first value is the name of the channel and the remaining values are those being output.

The clause [IN] for an input action  $u(\bar{x})^\ell$  proceeds by considering each local environment  $\vec{m}$  in turn; the corresponding variables are obtained as  $\mathbf{L}(\mathbf{B}.\ell)$ . It finds the value  $n$  of the input channel  $u$  according to  $\vec{m}$  and identifies the set of tuples  $\vec{o}$  that have been output over that channel according to  $\kappa$ . In case a tuple  $\vec{o}$  has the same length as the sequence of variables  $\bar{x}$  to be bound by the action, the existing local environment  $\vec{m}$  is extended with  $\vec{o}$  to create one of the new local environments  $\vec{m}\vec{o}$ , which is the result of the input; these are all recorded in  $\rho(\ell)$ . In case some tuple  $\vec{o}$  has a length that differs from that of  $\bar{x}$ , the corresponding channel  $n$  is a source of possible arity mismatch and  $n$  will be recorded in  $\psi$ .

The method used for forwarding local environments has some similarities to the one used in Section 3.3. In both cases we have new local environments for each labelled action; in Section 3.3 this was only the case for inputs, and hence no forwarding was needed for outputs or silent actions. Forwarding in both of these cases means copying (and possibly modifying) the local environment from the label preceding the current action to the label of the current action. The method used in this section is slightly different, however; in Section 3.3 we remembered the preceding label on the judgement itself, whereas here we make use of the information provided by  $\mathbf{B}$ . This is largely motivated by the more complex proof strategy needed for proving the relational analysis correct.

*Example 3.11.* Returning to Example 3.10 we have the following analysis result

$$\rho = \begin{array}{|c|c|c|c|c|c|c|} \hline \ell_\star & 1 & 2 & 3 & 4 & 5 & 6 \\ \hline \{\epsilon\} & \{\epsilon\} & \{\epsilon\} & \{nm, mn\} & \{nm, mn\} & \{nm, mn\} & \{nmn, mn m\} \\ \hline \end{array} \quad \kappa = \left\{ \begin{array}{l} c n m, \\ c m n, \\ n m, \\ m n \end{array} \right\} \quad \psi = \emptyset$$

This turns out to be the least analysis result that satisfies both  $\rho, \kappa \vdash_P \mathbf{P}_\star : \psi$  and  $\epsilon \in \rho(\ell_\star)$ .

To convince ourselves that this is correct let us consider the output action  $x\langle y \rangle^4$ . We have to check that

$$\rho(\mathbf{B}.4) \subseteq \rho(4)$$

$$\{\Pi_{x y @ \mathbf{L}(\mathbf{B}.4)}(\vec{m}) \mid \vec{m} \in \rho(\mathbf{B}.4)\} \subseteq \kappa.$$

The first inclusion clearly holds, since  $\mathbf{B}.4 = 3$  according to Example 3.10. To verify the second inclusion we first observe that  $\mathbf{L}(\mathbf{B}.4) = xy$  and since  $\rho(3) = \{nm, mn\}$ , we calculate  $\Pi_{xy @ xy}(nm) = nm$  and  $\Pi_{xy @ xy}(mn) = mn$  and thus the inclusion test amounts to checking  $\{nm, mn\} \subseteq \kappa$  which indeed holds.

Let us next consider the input action  $y(y)^6$ . Here we have to check the following two conditions

$$\begin{aligned} \{\vec{m}o \mid \vec{m} \in \rho(\mathbf{B.6}) \wedge n' = \Pi_{y@L.(\mathbf{B.6})}(\vec{m}) \wedge n'o \in \kappa\} &\subseteq \rho(6) \\ \{n' \mid \vec{m} \in \rho(\mathbf{B.6}) \wedge n' = \Pi_{y@L.(\mathbf{B.6})}(\vec{m}) \wedge n'\vec{o} \in \kappa \wedge |\vec{o}| \neq 1\} &\subseteq \psi. \end{aligned}$$

Let us focus on the first of them. From Example 3.10, we see that  $\mathbf{B.6} = 5$  and  $\mathbf{L.5} = xy$ . We know that  $\rho(5)$  contains the two sequences  $nm$  and  $mn$  and we now consider them one by one. In the case of  $nm$  the value of  $y$  ( $n'$  in the preceding clause) is  $n$  and since  $nm \in \kappa$  we get that  $o = m$  and we therefore have to check that  $nmn \in \rho(6)$  (which clearly holds). In the case of  $mn$  we make similar calculations, requiring us to check that  $mnm \in \rho(6)$ .

The constraints imposed by the other actions can be checked in a similar way. We shall return to the importance of requiring  $\epsilon \in \rho(\ell_\star)$  shortly.

*Correctness.* It is considerably more complex to prove the relational analysis correct than the previous analyses. Intuitively, this is due to the mismatch between the style of the static analysis and that of the semantics; the former uses explicit local environments whereas the latter uses a substitution-based semantics (rather than one with explicit environments). As shown in the scenario of Nielson et al. [2008] this means that one can sometimes find counterexamples to a straightforward generalization of the previous subject reduction results. Consequently we shall follow [Nielson et al. 2008] in showing that a more complex notion of analyzability is preserved under evaluation.

Let us fix the initial process  $P_\star$  of interest and define  $\mathbf{B}$ ,  $\mathbf{L}$ , and  $\ell_\star$  as shown in the preceding. We shall then define the following analyzability predicate

$$\rho, \kappa \Vdash_{P_\star}^{\mathbf{B}, \mathbf{L}} [Q] : \psi$$

to hold if and only if:

For each exposed subprocess  $\beta^\ell.Q'$  of  $[Q]$ , there exists a subprocess  $\alpha^\ell.P'$  of  $P_\star$  and a local environment  $\vec{m} \in \rho(\mathbf{B.}\ell)$ , corresponding to the variable sequence  $\mathbf{L.}(\mathbf{B.}\ell)$ , such that  $\beta^\ell.Q' \equiv (\alpha^\ell.P')[\vec{m}/\mathbf{L.}(\mathbf{B.}\ell)]$  and  $\rho, \kappa \vdash_P \alpha^\ell.P' : \psi$ .

Here a subprocess  $\beta^\ell.Q'$  is said to be *exposed* in  $Q$  if it is not prefixed by any actions; this ensures that  $\beta^\ell.Q'$  does not contain any free variables and this is essential for obtaining a congruence when replacing all variables in  $\mathbf{L.}(\mathbf{B.}\ell)$  with their values within  $\alpha^\ell.P'$ .

In Theorem 3.16 we shall establish a subject reduction result using this notion of analyzability. However, we shall first state a few auxiliary lemmas. The first two results clarify the properties of exposed subprocesses.

**LEMMA 3.12.** *Let  $P$  and  $Q$  be two processes such that  $P \equiv Q$ . For each exposed subprocess  $\alpha^\ell.P'$  of  $P$ , there exists an exposed subprocess  $\beta^\ell.Q'$  of  $Q$  such that  $\alpha^\ell.P' \equiv \beta^\ell.Q'$ .*

**LEMMA 3.13.** *If  $\alpha^\ell.P'$  is an exposed subprocess of  $P$  and  $\rho, \kappa \vdash_P [P] : \psi$  then  $\rho, \kappa \vdash_P [\alpha^\ell.P'] : \psi$ .*

The next auxiliary result shows how to establish analyzability of the process  $P_\star$  of interest—and it explains the role of the condition  $\epsilon \in \rho(\ell_\star)$ .

**LEMMA 3.14 INITIAL PROCESS.** *Let  $P_\star$ ,  $\mathbf{B}$ ,  $\mathbf{L}$  and  $\ell_\star$  be as shown above. If  $\rho, \kappa \vdash_P P_\star : \psi$  and  $\epsilon \in \rho(\ell_\star)$  then  $\rho, \kappa \Vdash_{P_\star}^{\mathbf{B}, \mathbf{L}} [P_\star] : \psi$ .*

Next we clarify how the structural congruence interacts with analyzability.

**LEMMA 3.15 STRUCTURAL CONGRUENCE.** *Let  $P$  and  $Q$  be two processes such that  $P \equiv Q$ . If  $\rho, \kappa \Vdash_P^{\star} [P] : \psi$  then  $\rho, \kappa \Vdash_P^{\star} [Q] : \psi$ .*

Finally, we are able to establish our subject reduction result.

**THEOREM 3.16 SUBJECT REDUCTION.** *If  $P \rightarrow Q$  and  $\rho, \kappa \Vdash_P^{\star} [P] : \psi$  then  $\rho, \kappa \vdash_{P^{\star}} [Q] : \psi$ .*

### 3.5 Bibliographical Notes

Many of the techniques for improving the precision of the analysis of Section 2 were originally developed for other calculi. As already mentioned, the techniques are to a large extent orthogonal and hence can be combined with each other; we shall return to this in Section 5.

The idea of checking for reachability is present in the development of analyses for the  $\lambda$ -calculus (see e.g. Nielson and Nielson [2002]). In Bodei et al. [1998, 2001a] we exploit it when analyzing the test construct  $[x = y]P$ : the continuation  $P$  should be analyzed only if the test  $x = y$  might succeed, that is, if  $\rho(x) \cap \rho(y) \neq \emptyset$ . In Bodei et al. [1999] we take this idea one step further and only require the continuation of an action to be analyzed if the action indeed may succeed—this is along the lines of the analysis in Table V.

The idea of determining the potential interaction points originates from Bodei et al. [2005], where logical addresses of subprocesses are used to estimate which subprocesses may interact—such pairs of addresses are said to be *compatible*. In Bodei et al. [2005], the logical addresses are obtained from an enhanced operational semantics. In Nielson et al. [2004] the same idea is developed for an analysis of BioAmbients but using a simple labelling schema to precompute which actions may occur in parallel branches that thereby may interact—just as we have seen in Table VI. The main point to notice is that only the skeleton of the actions are used; in particular we do not use information about the actual or canonical names involved (as they are not preserved under evaluation). More complex analyses are needed in order to capture this; see for example, Nielson and Nielson [2009]; Pilegaard et al. [2008]; Nielson and Nielson [2007b].

The idea of using localized environments goes back to the control flow analysis (CFA) of functional programs [Shivers 1988] and is presented in [Nielson et al. 1999] for the  $\lambda$ -calculus. Here contexts record the call structure of the various functions of the program; in the so-called  $k$ -CFA analyses up to  $k$  levels of such calls are recorded and it is possible to distinguish between the variable bindings in these contexts. As we have already seen, the important actions are now the input actions because they are the binders of variables. For the sake of simplicity we have only recorded one level of context information—the generalization to  $k$  levels is straightforward. Another generalization uses pairs of input and output labels as contexts.

The distinction between *independent attribute* analyses and *relational* analyses is classical in Static Analysis (see e.g. Nielson et al. [1999]). The actual analysis we have presented is based on Nielson et al. [2008], but performs a few simplifications; they are partly due to the  $\pi$ -calculus considered here being less complex than the pattern matching  $\pi$ -calculus in Nielson et al. [2008] and partly due to letting localized environments record the bindings after rather than before the labelled actions. A more complex development for a considerably more challenging language is performed in Bauer et al. [2008]. Our approach is purely syntax directed unlike the more powerful approaches of Venet [1998] and Feret [2002] that essentially require the process calculus

to be translated into the form of an abstract machine; as discussed in Nielson et al. [2008] and Bauer et al. [2008] we believe that this is likely to enable more researchers to perform static analyses that go beyond the simple independent attribute analyses.

#### 4. FLOW LOGIC AS A PROGRAM LOGIC

In this section we shall make the point that a Flow Logic is a Program Logic—in much the same way that a Hoare Logic [Apt 1981] is. We shall not give a formal definition of a Flow Logic because that would defeat our purposes, but we shall provide a set of normative guidelines based on more than a decade of research. The treatment in this section is not intended for the novice to Flow Logic, nor is it required for straightforward adaptations of Flow Logic to other calculi; for the novice who wants to read more, we recommend the development in the online Appendix.

##### 4.1 Flow Logic Judgements

Two notations parameterize a Flow Logic.

- (1) the programming notation being analyzed; and
- (2) the language used to express the imposed constraints.

In this article, we have so far focussed on programming notations that are process calculi while Nielson and Nielson [2002] has focussed on programming notations that are programming languages; we shall consider programming notations in more detail in Section 4.2. Similarly, the constraint language used has been a somewhat informal mathematical notation with ingredients from first order logic as well as inclusion constraints between analysis estimates; we shall consider the use of first order logic as a constraint language in Section 4.3.

For each syntactic category  $P$  of the programming notation, the Flow Logic defines an *analysis judgement* of the form

$$\bar{R} \vdash_{\mathbf{P}}^{\gamma, \Gamma} P : \bar{T}$$

for expressing the acceptability of the analysis information with respect to the syntactic entity in  $P$ . To be specific,  $P \in \mathbf{P}$  is a metavariable ranging over the syntactic category,  $\bar{R}$  is a (usually nonempty) list of analysis predicates expressing global analysis information,  $\bar{T}$  is a list of analysis predicates expressing local analysis information (and is often empty), and  $\gamma$  is an indicator of context information (and is often empty or absent).

The component  $\Gamma$  is an often empty or absent environment containing information of interest for the analysis. As an example it may contain information about higher-order syntactic entities; if our calculus is extended with a construct like  $\text{let } A(\tilde{x}) = P \text{ in } Q$ , that defines the higher-order abbreviation  $A$  for a possible recursive process  $P$ , that may be used within  $Q$ ; here we may take  $\Gamma(A) = (\tilde{x})P$  to record the presence of the definition. The  $\Gamma$  component may also be used for mapping identifiers in the programming notation to the elements in the domains of discourse—as for example associating a name with a representation of its scope.

It should be clear that the many analyses developed in the previous sections define analysis judgements as described here. As an example, the judgements  $\rho, \kappa \vdash_A^{\ell, X} \pi : \psi$  of Section 3.3 make use of all components except  $\Gamma$ .

##### 4.2 Programming Notations

One of the main ingredients of a Flow Logic is the programming notation. A programming notation, as for example a process calculus, is characterized by a number of



*syntactic categories*,  $P, P', \dots$ , (e.g. names, Name, actions, Act, and processes, Proc) as well a number of *syntactic formation rules*,

$$P ::= \sigma(P_1, \dots, P_n),$$

for creating syntactic terms; here each  $P_i \in \mathcal{P}_i$  is a meta-variable ranging over a syntactic category. As an example, we may have  $P ::= \pi.Q$  and  $\pi ::= u(x)$  where  $P, Q \in \text{Proc}$ ,  $\pi \in \text{Act}$  and  $u, x \in \text{Name}$ . We have chosen to formalize this as a context-free grammar where the nonterminals correspond to the syntactic categories and the productions correspond to the formation rules. Alternatively, one could have used a many-sorted algebra where the sorts correspond to the syntactic categories and the operators correspond to the formation rules.

In a given syntactic formation rule,  $P ::= \sigma(P_1, \dots, P_n)$ , it is common to define some positions as defining positions, together with an indication of their scope. As an example, for a rule like  $P ::= u(x).P$  the parameter  $x$  is a defining occurrence and its scope is just the continuation  $P$ ; in contrast,  $u$  is not a defining position.

As has been illustrated, we often annotate the syntactic terms with labels in order to more precisely identify subterms. This is also useful for providing context information that can be exploited in the analysis. For example in the construct  $\pi ::= u(x)^\ell$ , it might be useful to record that  $x$  is defined at label  $\ell$ .

A programming notation should also have a semantics. For process calculi some form of operational semantics is quite common but nothing prevents the use of other forms of semantics.

#### 4.3 First Order Logic as a Constraint Language

The other main ingredient of a Flow Logic is the language used to express the constraints imposed by the analysis. Here first order logic embodies a good part of the most commonly used mathematical notation needed and it will serve our purposes for the main part of this section.

A first order logic is usually characterized by a *universe of discourse*, which we shall model as an algebra  $\mathcal{U}$  (with just one sort). On top of this there are a number of *predicate symbols*; an  $n$ -ary predicate symbol  $R$  takes  $n$  parameters from the universe of discourse and yields a truth-value. The predicates are used to formalize the analysis domains of interest and example predicates from the previous sections are therefore  $\kappa$ ,  $\rho$  and  $\psi$ .

A *logical formula*  $\phi$  is built from the predicate symbols given their appropriate arguments (being terms of the algebra  $\mathcal{U}$ ), the usual logical connectives, and universal and existential quantification over variables ranging over elements of the universe of discourse. Sometimes we wish to be precise about the set of predicates allowed in a given formula; when only predicates in a given set  $\mathcal{R}$  are allowed, we shall say that the logical formula is based on the predicates in  $\mathcal{R}$ . Unless otherwise stated,  $\mathcal{R}$  will equal a fixed set  $\mathcal{R}_B$  of *base predicates* and this set should contain the analysis predicates  $\bar{R}$  and  $\bar{T}$  of the judgements  $\bar{R} \vdash_P^{\gamma, \Gamma} P : \bar{T}$  mentioned in Section 4.1.

We shall say that a logical subformula, e.g., an occurrence of a predicate, occurs at top-level in the formula  $\phi$ , if there are no explicit or implicit negations on the path to it; here an occurrence to the left of an implication is considered an implicit use of negation. More generally, we shall say that a logical subformula occurs positively if there is an even number of explicit or implicit negations on the path to it; clearly all top-level occurrences are also positive ones. A predicate is said to occur at top-level, or positively, whenever this holds for all its occurrences.

The semantics of first order logic is standard and will only be briefly summarized here. An interpretation  $\mathcal{I}$  assigns to each  $n$ -ary predicate  $R \in \mathcal{R}$  a subset  $\mathcal{I}(R)$  of  $\mathcal{U}^n$

Table XI. Syntax of Alternation-Free Least Fixed Point Logic (ALFP)

|                 |                                                                                                                                                                                                                               |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $v$             | $::= c \mid x \mid f(v_1, \dots, v_k)$                                                                                                                                                                                        |
| $\text{pre}$    | $::= R(v_1, \dots, v_k) \mid \neg R(v_1, \dots, v_k) \mid v_1 = v_2 \mid v_1 \neq v_2$<br>$\mid \text{pre}_1 \wedge \text{pre}_2 \mid \text{pre}_1 \vee \text{pre}_2 \mid \exists x : \text{pre} \mid \forall x : \text{pre}$ |
| $\text{clause}$ | $::= R(v_1, \dots, v_k) \mid \mathbf{1} \mid \text{clause}_1 \wedge \text{clause}_2$<br>$\mid \text{pre} \Rightarrow \text{clause} \mid \forall x : \text{clause}$                                                            |

(or alternatively a function  $\mathcal{I}_R : \mathcal{U}^n \rightarrow \{\text{true}, \text{false}\}$ ). The definition of validity of a closed formula  $\phi$ , i.e. one having no free variables, with respect to an interpretation  $\mathcal{I}$  is denoted by  $\mathcal{I} \models \phi$  and yields a truth-value (*true* or *false*). It is defined structurally in  $\phi$  also making use of a valuation mapping variables to elements of  $\mathcal{U}$  in order to deal with logical formulae that are not closed.

In the subsequent development we will occasionally need to use existential quantification from second order logic; in particular, existential quantification over predicates. We shall not detail the semantics of this as it is entirely standard and not of primary focus.

*Alternation-Free Least Fixed Point Logic.* We often find it useful to work with Alternation-Free Least Fixed Point Logic (abbreviated *ALFP*) [Nielson et al. 2002b]. This is a fragment of a first order logic that restricts the formation of logical formulae according to certain well-formedness criteria; it is a generalization of Horn clauses as well as Datalog [Apt et al. 1988; Chandra and Harel 1980] that has proved to have a number of properties essential for our development.

The syntax of ALFP is presented in Table XI, where we write  $c$  for analysis constants,  $x$  for analysis variables,  $v$  for analysis values,  $f$  for analysis functions,  $R$  for analysis predicates,  $\text{pre}$  for preconditions, and  $\text{clause}$  for clauses.

The clauses are interpreted over a universe  $\mathcal{U}$  of analysis constants; indeed,  $c$  is an element of  $\mathcal{U}$ ,  $f$  has arity  $\mathcal{U}^n \rightarrow \mathcal{U}$  (for some  $n$ ), and  $x$  ranges over  $\mathcal{U}$ . The interpretation is given in terms of satisfaction relations

$$(\mathcal{I}, \iota) \models \text{pre} \text{ and } (\mathcal{I}, \iota) \models \text{clause},$$

where  $\mathcal{I}$  is an interpretation of analysis predicates,  $\iota$  is an interpretation of analysis variables, which we extend to operate on analysis values. The definition is standard and is shown in Table XII.

We shall often need to demand that the clauses in ALFP are *stratified*. This intuitively means that no predicate depends on the negation of itself. We refer to [Nielson et al. 2002b] for the details.

#### 4.4 First Order Flow Logic Specifications

A First Order Flow Logic  $\mathcal{F}$  is a program logic relative to a choice of a programming notation, e.g. a process calculus, and a fragment of a first order logic, e.g. ALFP, for expressing the constraints between analysis predicates.

As already explained, for each syntactic category,  $P$ , the Flow Logic defines an<sup>1</sup> *analysis judgement* of the form

$$\bar{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} P : \bar{T}$$

<sup>1</sup>It simplifies the presentation to assume that there is just one analysis judgement for each syntactic category, but it is not an essential assumption.

Table XII. Interpretation of Alternation-Free Least Fixed Point Logic (ALFP)

|                                                                       |                                                                                                       |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| $(\mathcal{I}, \iota) \models R(v_1, \dots, v_k)$                     | iff $(\iota(v_1), \dots, \iota(v_k)) \in \mathcal{I}(R)$                                              |
| $(\mathcal{I}, \iota) \models \neg R(v_1, \dots, v_k)$                | iff $(\iota(v_1), \dots, \iota(v_k)) \notin \mathcal{I}(R)$                                           |
| $(\mathcal{I}, \iota) \models v_1 = v_2$                              | iff $\iota(v_1) = \iota(v_2)$                                                                         |
| $(\mathcal{I}, \iota) \models v_1 \neq v_2$                           | iff $\iota(v_1) \neq \iota(v_2)$                                                                      |
| $(\mathcal{I}, \iota) \models \text{pre}_1 \wedge \text{pre}_2$       | iff $(\mathcal{I}, \iota) \models \text{pre}_1$ and $(\mathcal{I}, \iota) \models \text{pre}_2$       |
| $(\mathcal{I}, \iota) \models \exists x : \text{pre}$                 | iff $(\mathcal{I}, \iota[x \mapsto a]) \models \text{pre}$ for some $a \in \mathcal{U}$               |
| $(\mathcal{I}, \iota) \models \forall x : \text{pre}$                 | iff $(\mathcal{I}, \iota[x \mapsto a]) \models \text{pre}$ for all $a \in \mathcal{U}$                |
| <hr/>                                                                 |                                                                                                       |
| $(\mathcal{I}, \iota) \models R(v_1, \dots, v_k)$                     | iff $(\iota(v_1), \dots, \iota(v_k)) \in \mathcal{I}(R)$                                              |
| $(\mathcal{I}, \iota) \models 1$                                      | iff true                                                                                              |
| $(\mathcal{I}, \iota) \models \text{clause}_1 \wedge \text{clause}_2$ | iff $(\mathcal{I}, \iota) \models \text{clause}_1$ and $(\mathcal{I}, \iota) \models \text{clause}_2$ |
| $(\mathcal{I}, \iota) \models \text{pre} \Rightarrow \text{clause}$   | iff $(\mathcal{I}, \iota) \models \text{clause}$ whenever $(\mathcal{I}, \iota) \models \text{pre}$   |
| $(\mathcal{I}, \iota) \models \forall x : \text{clause}$              | iff $(\mathcal{I}, \iota[x \mapsto a]) \models \text{clause}$ for all $a \in \mathcal{U}$             |

for expressing the acceptability of analysis information with respect to syntactic entities. For each syntactic formation rule  $P ::= \sigma(P_1, \dots, P_n)$  the Flow Logic defines exactly one<sup>2</sup> clause of the form

$$\vec{R} \vdash_{\vec{P}, \Gamma}^{\gamma} \sigma(P_1, \dots, P_n) : \vec{T} \text{ iff } \phi,$$

where  $\phi$  is a logical formula in the fragment of first order logic considered and that is based on a set of predicates  $\mathcal{R}$  to be detailed in the following. The previous sections give several examples of such clauses—see Tables III, V, VII, VIII and X.

The set  $\mathcal{R}$  of predicates used on the right-hand side of these clauses, that is in  $\phi$ , contains two different kinds of predicates. It can be a base predicate from the set  $\mathcal{R}_B$  already mentioned in Section 4.3; recall that we assume that this set contains all of the predicates  $\vec{R}$  and  $\vec{T}$ . Actually,  $\phi$  may use predicates in  $\mathcal{R}_B \setminus \{\vec{R}, \vec{T}\}$  and they will implicitly be existentially quantified (using quantifiers of second order logic)—special care needs to be taken in the subsequent development.

The set  $\mathcal{R}$  of predicates used on the right-hand side of the clauses may also contain analysis judgement corresponding to one of the syntactic categories of the programming notations; we shall write  $\mathcal{R}_J$  for these predicates. They have the general form

$$\vec{R}' \vdash_{\vec{P}', \Gamma'}^{\gamma'} \sigma'(P_1, \dots, P_n) : \vec{T}',$$

and will be subject to the following conditions.

- (1)  $\sigma'(P_1, \dots, P_n)$  is often some of the  $P_i$  but may be a more complex syntactic term.
- (2)  $\vec{R}'$  is usually  $\vec{R}$  or a subsequence of  $\vec{R}$ .
- (3)  $\gamma'$  is usually  $\gamma$  or is obtained from  $\gamma$  using information in  $\sigma'(P_1, \dots, P_n)$ ,  $\vec{R}$ ,  $\vec{T}$  or  $\Gamma$ .
- (4)  $\Gamma'$  is usually  $\Gamma$  or an extension of it where some of the defining positions of  $\sigma'(P_1, \dots, P_n)$  give rise to new entries.
- (5)  $\vec{T}'$  is often empty or is otherwise some sequence of base predicates in  $\mathcal{R}_B \setminus \{\vec{R}\}$ .

It is easy to check that the clauses of Tables III, V, VII, VIII and X satisfy these conditions.

To distinguish between various formats of Flow Logic the following terminology has been introduced [Nielson and Nielson 2002].

<sup>2</sup>This is essential and differs from Type Systems where in general, there will be at least one axiom scheme or inference rule defined for each syntactic formation rule.

- A Flow Logic is *compositional* whenever all formulae of the form  $\phi$  only contain judgements where the syntactic component is a metavariable (and not a more complex term); it is *abstract* otherwise.
- A Flow Logic is *verbose* whenever all predicate sequences  $\vec{T}$  of all judgements are empty; it is *succinct* otherwise.

The Flow Logics presented in this article are all compositional and succinct.

#### 4.5 Well-Definedness

So far our treatment of a First Order Flow Logic  $\mathcal{F}$  has been purely syntactic. Clearly it is intended to have a meaning and to enjoy some properties useful for static analysis. One of the fundamental properties of Flow Logic, which it shares with Type Systems, is that there is a clear separation between determining:

- whether or not a given analysis estimate  $\vec{R}, \vec{T}$  is an acceptable description of a program  $P$ ; and
- finding the best such description, usually the least acceptable description with respect to some partial order.

In Type Systems, the first property is known as type checking and the second as type inference. In Flow Logic, the first property ensures the well-definedness of the judgements  $\vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} P : \vec{T}$ , i.e., that they denote either true or false, and this is dealt with in this subsection, while the second property is dealt with under the considerations of the Moore Family property in Section 4.8.

Assume that we have a judgement  $\vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} P : \vec{T}$ , a program fragment  $\sigma(P_1, \dots, P_n)$  of the corresponding syntactic category, and an interpretation  $\mathcal{I}$  of  $\mathcal{R}_B$  (or at least of  $\{\vec{R}, \vec{T}\}$ ). We shall then define what it means for  $\mathcal{I}(\vec{R}, \vec{T})$  to be an *admissible analysis estimate* for  $\sigma(P_1, \dots, P_n)$  according to the Flow Logic  $\mathcal{F}$ .

**Definition 4.1 (Admissibility of Analysis Estimates).** The interpretation  $\mathcal{I}$  is admissible for  $\vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} \sigma(P_1, \dots, P_n) : \vec{T}$ , written

$$\mathcal{I} \text{ sat } \vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} \sigma(P_1, \dots, P_n) : \vec{T},$$

whenever there exists an interpretation  $\mathcal{J}$  of  $\mathcal{R}_B \cup \mathcal{R}_J$  such that

- $\mathcal{J}(\vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} \sigma(P_1, \dots, P_n) : \vec{T}) = \text{true}$ ;
- $\mathcal{J}(R) = \mathcal{I}(R)$  for all  $R \in \mathcal{R}_B$  (abbreviated  $\mathcal{J} \upharpoonright_{\mathcal{R}_B} = \mathcal{I}$ );
- $\mathcal{J} \text{ SAT } \mathcal{F}$ ;

where the latter condition expresses that the interpretation  $\mathcal{J}$  satisfies the Flow Logic  $\mathcal{F}$  as defined by the following.

**Definition 4.2 (Satisfaction of Flow Logic).** The interpretation  $\mathcal{J}$  satisfies the Flow Logic  $\mathcal{F}$ , written

$$\mathcal{J} \text{ SAT } \mathcal{F}$$

whenever for each clause  $\vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} \sigma(P_1, \dots, P_n) : \vec{T}$  iff  $\phi$  in  $\mathcal{F}$ , and for each program  $\sigma(P'_1, \dots, P'_n)$  in the syntactic category:

$$\mathcal{J}(\vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} \sigma(P'_1, \dots, P'_n) : \vec{T}) \iff \mathcal{J} \models (\phi[P'_1/P_1 \dots P'_n/P_n]).$$

Thus Definition 4.1 merely says that  $\mathcal{J}$  satisfies all clauses in  $\mathcal{F}$ . Definition 4.2 is particularly pleasant to work with if there is an optimal way of constructing  $\mathcal{J}$  from  $\mathcal{I}$ ; we explore this in the following.

*Definition 4.3 (Well-Defined Flow Logic).* We shall say that  $\mathcal{F}$  is *well-defined* whenever  $\llbracket \mathcal{F}, \mathcal{I} \rrbracket \text{ SAT } \mathcal{F}$  for all interpretations  $\mathcal{I}$ ; here  $\llbracket \mathcal{F}, \mathcal{I} \rrbracket$  is the interpretation defined by

$$\llbracket \mathcal{F}, \mathcal{I} \rrbracket = \bigsqcup \{ \mathcal{J} \mid \mathcal{J} \text{ SAT } \mathcal{F} \wedge \mathcal{J} \upharpoonright_{\mathcal{R}_B} = \mathcal{I} \}.$$

We then have the following.

**THEOREM 4.4.** *For a well-defined Flow Logic  $\mathcal{F}$  the condition  $\mathcal{I} \text{ sat } \vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} P : \vec{T}$  is equivalent to  $\llbracket \mathcal{F}, \mathcal{I} \rrbracket \left( \vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} P : \vec{T} \right)$ .*

*Two Approaches to Well-Definedness.* To show that the Flow Logic  $\mathcal{F}$  is well-defined, it is sufficient to ensure that the set  $\{ \mathcal{J} \mid \mathcal{J} \text{ SAT } \mathcal{F} \wedge \mathcal{J} \upharpoonright_{\mathcal{R}_B} = \mathcal{I} \}$  is a singleton set. This means that there is only one way to extend  $\mathcal{I}$  to an interpretation  $\mathcal{J}$  for the Flow Logic  $\mathcal{F}$ . This is the case whenever the following holds

For all judgements  $\vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} P : \vec{T}$  of  $\mathcal{F}$  and for all concrete programs  $P'$  of the corresponding syntactic category, the formula  $\vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} P' : \vec{T}$  can be finitely unfolded (using the clauses of  $\mathcal{F}$ ) to a formula  $\phi$  that is based on  $\mathcal{R}_B$  only.

The notion of unfolding is as in Table IV and the previous procedure succeeds whenever  $\mathcal{F}$  is a compositional Flow Logic. It also succeeds when the syntactic parts of judgements on the right-hand sides of clauses are proper subformulae of the ones on the left hand sides—and even more generally, whenever they are strictly smaller in some well-founded order.

Another approach to ensuring well-definedness is to rely on fixed point theory. The Flow Logic  $\mathcal{F}$  and an interpretation  $\mathcal{I}$  can be seen as defining an operator  $\llbracket \mathcal{F}, \mathcal{I} \rrbracket$  over the complete lattice of interpretations; it is given by

$$\llbracket \mathcal{F}, \mathcal{I} \rrbracket(\mathcal{J}) = \mathcal{J}',$$

where

$$\mathcal{J}'(R) = \begin{cases} \mathcal{I}(R) & \text{if } R \in \mathcal{R}_B \\ \mathcal{J} \models \phi[P'_1/P_1 \dots P'_n/P_n] & \text{if } R \text{ is } \vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} \sigma(P'_1, \dots, P'_n) : \vec{T} \text{ and} \\ & \mathcal{F} \text{ contains } \vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} \sigma(P_1, \dots, P_n) : \vec{T} \text{ iff } \phi \end{cases}$$

This definition is unambiguous because  $\mathcal{F}$  contains exactly one clause with a left hand side involving the syntactic operator  $\sigma$ . It follows that  $\mathcal{J} \text{ SAT } \mathcal{F} \wedge \mathcal{J} \upharpoonright_{\mathcal{R}_B} = \mathcal{I}$  is equivalent to  $\mathcal{J} = \llbracket \mathcal{F}, \mathcal{I} \rrbracket(\mathcal{J})$ , and hence that  $\llbracket \mathcal{F}, \mathcal{I} \rrbracket(\mathcal{J}) = \bigsqcup \{ \mathcal{J} \mid \mathcal{J} = \llbracket \mathcal{F}, \mathcal{I} \rrbracket(\mathcal{J}) \}$  is the least upper bound of all fixed points of  $\llbracket \mathcal{F}, \mathcal{I} \rrbracket(\mathcal{J})$ . To ensure that this is itself a fixed point, it suffices to ensure that  $\llbracket \mathcal{F}, \mathcal{I} \rrbracket$  is monotonic because then Tarski's fixed point theorem [Tarski 1955] ensures the existence of a complete lattice of fixed points and in particular a greatest fixed point. Monotonicity of  $\llbracket \mathcal{F}, \mathcal{I} \rrbracket$  is ensured when all judgements (predicates of  $\mathcal{R}_J$ ) occurring on the right-hand side of the clauses of the Flow Logic occur in positive positions only. This is known as a *coinductive* definition and intuitively says that interpretations are only rejected if they explicitly violate the Flow Logic (as opposed to being admitted if they explicitly satisfy the Flow Logic). As an example, if the specification is extremely underspecified to the extent of saying that

$$\vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} \sigma(P_1, \dots, P_n) : \vec{T} \text{ iff } \vec{R} \vdash_{\mathcal{P}}^{\gamma, \Gamma} \sigma(P_1, \dots, P_n) : \vec{T},$$

then the coinductive interpretation admits all analysis estimates, whereas the inductive interpretation (corresponding to the least fixed point) would admit no analysis estimate.

The two approaches actually coincide in the sense that they both designate the greatest fixed point of a functional and hence both can be viewed as coinductive definitions. In the first approach this is obvious because there only is one fixed point, and in the second approach it is due to Tarski's fixed point theorem [Tarski 1955]. This is one place where our approach differs from that of Type Systems, which generally favor the inductive rather than the coinductive interpretation of clauses. For process calculi, coinductive definitions may be needed when there are higher-order features, such as processes that can be communicated over channels, or named process constants as in let  $A(\vec{x}) = P$  in  $Q$ , whereas for simpler process calculi using replication instead of recursion (like the  $\pi$ -calculus), one can usually dispense with coinduction.

#### 4.6 The Subject Reduction Result

So far in this section we have ignored the semantics of our programming notation. Clearly the acceptability of analysis estimates—interpretations  $\mathcal{I}$ —must be related to the semantics in a suitable way. For process calculi it is customary to use a semantics based on (possibly labelled) transition systems, perhaps in the form of a structural operational semantics, perhaps in the form of a reaction semantics, and possibly the semantics is based on a structural congruence.

Where there is a structural congruence,  $P_1 \equiv P_2$ , we usually want to ensure that structurally congruent processes cannot be differentiated from the point of view of the static analysis. This would suggest that we should establish the following property.

$$\text{If } P_1 \equiv P_2 \text{ then } \mathcal{I} \text{ sat } \vec{R} \vdash_{\mathbf{p}}^{\gamma, \Gamma} P_1 : \vec{T} \Leftrightarrow \mathcal{I} \text{ sat } \vec{R} \vdash_{\mathbf{p}}^{\gamma, \Gamma} P_2 : \vec{T}.$$

However, this may be problematic because the structural congruence usually makes use of  $\alpha$ -renaming of names while the Flow Logic incorporates names into the predicates constituting the analysis estimate; hence the property does not hold as it stands. Our proposed solution is to view the countably infinite set of names as being a countable union of pairwise disjoint sets of countably infinite names each having a canonical representative; the canonical representative for the set of names in which  $n$  is a member is denoted  $[n]$ . Without loss of generality, the structural congruence is modified so as to use *disciplined  $\alpha$ -renaming* (e.g. Bodei et al. [2001b]), where  $n$  can only be replaced by  $m$  if  $[n] = [m]$ . It is then possible to ignore the issue in the Flow Logic specification itself by simply ensuring that it is only invoked on syntactic elements where all names have been replaced by their canonical representatives. We have written  $[P]$  for the result of renaming  $P$  in this way and the property then reads:

*Property 4.5 (Structural Congruence).* Assume that  $P_1 \equiv P_2$ . Then  $\mathcal{I} \text{ sat } \vec{R} \vdash_{\mathbf{p}}^{\gamma, \Gamma} [P_1] : \vec{T}$  if and only if  $\mathcal{I} \text{ sat } \vec{R} \vdash_{\mathbf{p}}^{\gamma, \Gamma} [P_2] : \vec{T}$ .

Lemma 2.4 is an instance of this property, whereas Lemma 3.15 is a variation of it. The proof is usually by induction on the proof tree establishing  $P_1 \equiv P_2$ .

For an unlabelled transition system,  $P_1 \longrightarrow P_2$ , the semantic correctness statement often takes the form of a *subject reduction result* as borrowed from Type Systems. This merely says that the analysis information is preserved under the semantics:

*Property 4.6 (Subject Reduction).* If  $P_1 \longrightarrow P_2$  and  $\mathcal{I} \text{ sat } \vec{R} \vdash_{\mathbf{p}}^{\gamma, \Gamma} [P_1] : \vec{T}$  then  $\mathcal{I} \text{ sat } \vec{R} \vdash_{\mathbf{p}}^{\gamma, \Gamma} [P_2] : \vec{T}$ .

Theorems 2.6, 3.2, and 3.9 are all instances of this property, whereas Theorems 3.4 and 3.16 are variations of it. The proof is usually by induction on the proof tree establishing  $P_1 \longrightarrow P_2$ .



For a labelled transition system, we need to add information about the relationship of the labels to the analysis information. In the case of output labels this usually gives rise to extra conclusions in the subject reduction result. In the case of input labels (in particular input from the environment as in the case of open systems), this usually gives rise to extra assumptions in the subject reduction result.

#### 4.7 The Adequacy Result

Semantic soundness of an analysis is often seen as composed of two components. One is the subject reduction result saying that the information is preserved during evaluation as discussed in the preceding. The other is an adequacy result saying that the analysis provides a decidable approximation (the *static* property) to a usually undecidable property expressed directly in terms of the semantics (the *dynamic* property). Clearly the details greatly depend on the applications that one has in mind, ranging from identifying dead code to ensuring the correct delivery of messages.

In this article we have given several examples of adequacy results, namely Theorems 2.7, 2.8, and 2.9. Often they take the form of using an error component among the  $\vec{T}$  in  $\vec{R} \vdash_{\vec{P}}^{\gamma, \Gamma} P : \vec{T}$  (e.g.  $\psi$ ). The error component is used to report any offending behavior as regards the static property. The adequacy result then states that an empty error component ensures the dynamic property.

We have experimented with a different way of working with error components [Nielson and Nielson 2007a] (see also Bodei et al. [2007]). This has taken the form of working with a variation of Alternation-Free Least Fixed Point Logic, where predicates can be annotated with exclamation marks in some of their positive occurrences. For each predicate  $R$  we then have an error predicate  $E_R$  and whenever the Flow Logic contains a literal  $R!(\vec{u})$  it is treated as standing for  $\neg R(\vec{u}) \Rightarrow E_R(\vec{u})$  (possibly incorporating additional entities from the context). This results in a more prescriptive flavor of Flow Logics than the usual descriptive one—from a technical point of view it is essential to make use of stratification, as discussed in Section 4.8 when dealing with the Moore Family property.

#### 4.8 The Moore Family Result

Just as Flow Logic borrows a lot from Type Systems in the distinction between acceptability of analysis estimates and the computation of best analysis estimates, it also borrows a lot from Abstract Interpretation regarding how to characterize best analysis estimates. Analysis estimates  $\mathcal{I}(\vec{R}, \vec{T})$  are elements of a complete lattice and we want to ensure that there is a least (and hence unique) analysis estimate that is admissible for a process  $P$  with respect to the Flow Logic  $\mathcal{F}$ .

We formulate this as the property that the set of admissible analysis estimates  $\mathcal{A}_{\gamma, \Gamma, P} = \{\mathcal{I}(\vec{R}, \vec{T}) \mid \mathcal{I} \text{ sat } \vec{R} \vdash_{\vec{P}}^{\gamma, \Gamma} P : \vec{T}\}$  constitutes a Moore Family for all choices of  $P$ ,  $\gamma$ , and  $\Gamma$ . Recall that a Moore Family (e.g. Nielson et al. [1999]) is a set  $\mathcal{A}$  such that  $\sqcap Y \in \mathcal{A}$  whenever  $Y \subseteq \mathcal{A}$ ; consequently  $\sqcap \mathcal{A}$  will be the desired analysis estimate. This is a mere proof of existence—the situation in which it is computable is dealt in Section 4.9. In the world of type systems this corresponds to the existence of *principal typings*.

*Property 4.7 (Moore Family).*  $\{\mathcal{I}(\vec{R}, \vec{T}) \mid \mathcal{I} \text{ sat } \vec{R} \vdash_{\vec{P}}^{\gamma, \Gamma} P : \vec{T}\}$  is a Moore Family for all choices of  $P$ ,  $\gamma$  and  $\Gamma$ .

Proposition 2.10 is an instance of this property.

The Moore Family property does not hold for arbitrary first order flow logics. However, in the same way that a semantics-directed approach to the formulation of the Flow Logic greatly facilitates the ease with which a correct Flow Logic can be developed, the use of appropriate fragments of First Order Logic greatly facilitates the ease

with which a Flow Logic satisfying the Moore Family property can be developed. It is here we have found the use of Alternation-free Least Fixed Point Logic, ALFP (see Section 4.3), very useful: when the clauses are expressed in ALFP the Moore Family property is usually straightforward to prove.

The proof of the Moore Family property generally follows the same proof strategy showing that  $\vec{R} \vdash_{\vec{P}}^{\gamma, \Gamma} P : \vec{T}$  was well-defined in the first place. In the case of compositional definitions this is a straightforward inductive proof; in other cases some kind of well-founded induction might suffice, whereas, in the case of coinductive definitions, a proof by coinduction is called for.

It is possible to formulate large parts of Abstract Interpretation, in particular the considerations of Galois connections in this setting, but we shall refrain from embarking upon this here (e.g. Hansen et al. [1999]).

#### 4.9 Implementation

Whenever possible, perhaps after several transformations on the Flow Logic specification, we would like to obtain a specification such that:

- (1) it stays within the fragment of First Order Logic known as ALFP (as motivated in the preceding);
- (2) it is *compositional* (so that no coinduction is needed in order to ensure well-definedness);
- (3) it is *verbose* (so that there are no existentially quantified second-order predicates—or they arise in a very benign manner only);
- (4) it uses a flat universe of discourse whose size is proportional to the program being analyzed.

In this case the analysis of a program gives rise to a formula in ALFP over a flat universe of discourse. The formula is linear in the size of the program analyzed (viewing the size of the Flow Logic itself as being of constant size since it is fixed for each analysis performed).

This approach gives us a number of benefits. One is that there is a very direct algorithm for computing the least solutions. Our own Succinct Solver [Nielson et al. 2002b] can deal with all of ALFP, while packages such as Datalog and XSB Prolog can deal with fragments of ALFP that still extend Horn clauses.

Another main benefit is that there is no complexity gap between verifying whether or not  $\mathcal{I}(\vec{R}, \vec{T})$  is an admissible analysis estimate and computing the least, admissible analysis estimate [McAllester 1999]. This is unlike the situation in Type Systems, where often type checking is doable in polynomial time whereas type inference may take exponential time (or be hard for nondeterministic polynomial time).

A final benefit is that it is very easy to give a useful upper bound on the worst-case time complexity. If the program  $P$  is of size  $n$  the logical formula will often be of size  $O(n)$  as will the universe of discourse. In this case the computational complexity of either of the two problems discussed in the preceding is bounded by  $O(n^{r+1})$ , where  $r$  is the maximal nesting depth of quantifiers in the logical formula generated for the judgement  $\vec{R} \vdash_{\vec{P}}^{\gamma, \Gamma} P : \vec{T}$  for the program  $P$ . While  $r$  may in the worst-case be proportional to  $n$ , it is often a small constant. This is the case whenever the Flow Logic specification contains no definition whose right hand side contains any judgements within the scope of a quantifier; in this case  $r$  can be determined as the maximum nesting depth of quantifiers on the right-hand sides of clauses in the Flow Logic. This is the case for all the Flow Logics of this article as can easily be seen by inspecting Tables III, V, VII, VIII and X. The reason is that each occurrence of  $\forall i \in I$  in the clauses named [SUM] merely is a shorthand for the clause to be generated for  $\Sigma_{i \in I} \pi_i.P_i$ .

*Property 4.8 (Polynomial Time Analysis).* Given a Flow Logic  $\mathcal{F}$  fulfilling conditions (1)–(4) and a program  $P$  and universe  $\mathcal{U}$ , both of size  $n$ , assume that the maximum nesting depth of quantifiers in any clause of  $\mathcal{F}$  is  $r$  and that no clause contains any judgements within the scope of a quantifier. Then the solution guaranteed by Theorem 4.7 can be found in time  $O(n^{r+1})$ .

For the simple class of context independent analyses illustrated in Section 2,  $r$  takes the value 2, giving an overall cubic time bound on the complexity. We refer to Nielson et al. [2002b] for an example proof of this property.

#### 4.10 Pragmatics

As is clear from the development in the previous sections it is helpful to allow a few abbreviations that can all be expanded into first order logic as part of the algorithm in Table IV. We often write  $\vec{v} \in R$  to denote  $R(\vec{v})$ . When testing for intersection we write  $R_1(\vec{v}) \cap R_2(\vec{v}') \neq \emptyset$  to denote  $\exists \vec{v} : R_1(\vec{v}, \vec{v}) \wedge R_2(\vec{v}', \vec{v})$ . When testing for difference we write  $R_1(\vec{v}) \setminus R_2(\vec{v}') \neq \emptyset$  to denote  $\exists \vec{v} : R_1(\vec{v}, \vec{v}) \wedge \neg R_2(\vec{v}', \vec{v})$ . When expressing subset relationships we write  $R_1(\vec{v}) \subseteq R_2(\vec{v}')$  to denote  $\forall \vec{v} : R_1(\vec{v}, \vec{v}) \Rightarrow R_2(\vec{v}', \vec{v})$ . This may involve unions, e.g.,  $R_1(\vec{v}) \cup R_2(\vec{v}') \subseteq R_3(\vec{v})$ , which denotes  $\forall \vec{w} : R_1(\vec{v}, \vec{w}) \vee R_2(\vec{v}', \vec{w}) \Rightarrow R_3(\vec{v}, \vec{w})$ , and may also involve intersections, e.g.,  $R_1(\vec{v}) \cap R_2(\vec{v}') \subseteq R_3(\vec{v})$  which denotes  $\forall \vec{w} : R_1(\vec{v}, \vec{w}) \wedge R_2(\vec{v}', \vec{w}) \Rightarrow R_3(\vec{v}, \vec{w})$ . A more recent abbreviation [De Nicola et al. 2010] is to write

$$\begin{aligned} R_1[R_2] &= \bigcup_{\vec{v} \in R_2} R_1(\vec{v}) = \{\vec{w} \mid \exists \vec{v} \in R_2 : R_1(\vec{v}, \vec{w})\} \\ R_1(R_2) &= \bigcap_{\vec{v} \in R_2} R_1(\vec{v}) = \{\vec{w} \mid \forall \vec{v} \in R_2 : R_1(\vec{v}, \vec{w})\}. \end{aligned}$$

It is worth pointing out that this allows their use in inclusions and that they can usually be expanded away as follows:  $R_1[R_2] \subseteq R_3$  is equivalent to  $\forall \vec{v} \in R_2 : R_1(\vec{v}) \subseteq R_3$ , and  $R_1 \subseteq R_2(R_3)$  is equivalent to  $\forall \vec{v} \in R_3 : R_1 \subseteq R_2(\vec{v})$ . (Clearly the inclusions can be expanded away as well.)

Clearly this account of First Order Flow Logic can be generalized in many ways. The first order logic can have a multisorted universe of discourse. The first order logic itself could be multisorted, e.g. to distinguish between positive and negative positions (as is in fact the case in ALFP); the first order logic could be replaced by a second order logic (in extension of our implicit use of second order existential quantification for succinct Flow Logics) as might be needed to deal with so-called pathway analyses [Nielson and Nielson 2009; Pilegaard et al. 2008]; we might allow more general complete lattices than the powersets in which the relations take their values. Finally, the first order logic can be replaced by a modal logic [Bolander and Hansen 2007]. The list is endless. However, it is important that we retain the desirable properties of Flow Logic: well-definedness, subject reduction, adequacy, Moore Family result, and a notion of implementability.

## 5. CONCLUSION

Flow Logic started out as an approach to the static analysis of programming languages that would be able to integrate key elements of the approaches of Data Flow Analysis, Constraint Based Analysis (in particular the propagation of constraints as expressible using inclusions or implications) and Abstract Interpretation (in particular the Moore Family property) while taking an approach sharing the philosophy of Type Systems (in particular the distinction between type checking and type inference). As illustrated in a number of papers (surveyed in Nielson and Nielson [2002]), this has allowed the

development of a number of static analyses for a variety of language paradigms (including imperative, functional, object oriented, concurrent, and distributed and mobile features) in a rather succinct way, while at the same time opening up for transferring analysis ideas between language paradigms—the need for which is illustrated by the many places where Nielson et al. [1999] reports on independent discovery of the same basic analysis idea among different programming language communities. The logical format used for presenting specifications focuses on ensuring the implementability of the analyses—often in low polynomial (cubic) time—while ensuring semantic correctness, which often takes the form of subject reduction and adequacy results. The relationship between Flow Logic and Type Systems is studied in De Nicola et al. [2008, 2010] and the relationship between Flow Logic and Model Checking of logics in the CTL family is studied in Nielson and Nielson [2010].

Looking beyond programming languages, quite a number of papers have developed Flow Logic for the  $\pi$ -calculus; the aim was to establish various security properties using the information obtainable by static analysis. In Section 2 we covered the basic context independent approach, while illustrating the key points of Flow Logic. Various ways of dealing with context were then considered in Section 3: reachability; which actions can interact with each other; separating different defining occurrences of the same name; and taking the relation between the values of variables into account. Subsequently we have been able to transfer our insights from analyzing the  $\pi$ -calculus to a variety of other process calculi embodying other computational paradigms. We shall survey some of these developments in the following.

Distributed and mobile processes have been considered in the context of Mobile Ambients [Cardelli and Gordon 2000], where all computational processes are encapsulated in so-called ambients that may nest and move. In Hansen et al. [1999], best described in Nielson et al. [2002], it is demonstrated how a Flow Logic can be used to validate the security properties of a firewall modelled in this setting. Extensions of this work considered Discretionary Ambients [Nielson et al. 2005] and showed how to deal with mandatory access control in a setting that incorporates both a Bell-LaPadula model for confidentiality as well as a Biba model for integrity. Finally, a number of Flow Logics have been formulated in the context of the BioAmbients calculus [Regev et al. 2004] in order to analyze reachability properties of biological systems [Nielson et al. 2004; Pilegaard et al. 2006b].

Flow Logics for other aspects of code mobility have been developed in the context of the Kernel Language for Agents Interaction and Mobility, KLAIM [De Nicola et al. 1998], where both data and processes have mobility over a network of localized tuple-spaces. In Tolstrup et al. [2007], it was shown how to ensure that systems conform to a notion of locality based security policies and in Probst et al. [2007] it was shown how to assess the security threat imposed by an organization insider. A notion of sandboxing has been considered in Hansen et al. [2006] that addresses a version of KLAIM where remote process invocation is governed by a security policy to be enforced on the remote invocation, and the paper shows how a Flow Logic can ensure secure sandboxing. This was further extended in Hansen et al. [2008], which developed a Flow Logic for validating the conformance of client software with respect to a license conformance policy. A comparison between Flow Logic and Type Systems for a dialect of Klaim can be found in De Nicola et al. [2010].

Flow Logics for the analysis of cryptographic protocols and regular term languages were first considered in the context of the Spi calculus [Abadi 1999a], which enriches the  $\pi$ -calculus with a cryptographic term language. In this context it was shown how to use Flow Logic for analyzing cryptographic protocols in cubic time [Nielson et al. 2002a]. Later work addressed the  $\nu$ Spi calculus, an extension of Spi where each instance of the same encryption is guaranteed to yield a different ciphertext

corresponding to the practical notion of confounders [Abadi 1999b]. Bodei et al. [2002] showed how to adapt the Flow Logic to this setting while incorporating an attacker model of Dolev-Yao strength. This was followed by a number of contributions that addressed the LySa calculus [Bodei et al. 2003], a Spi-inspired construction that abandons the notion of named channels in favor of a single global ether and allows annotation of points of origin and destination for messages. The Flow Logics developed in Bodei et al. [2003, 2005] addressed both cryptographically protected secrecy and authentication.

## ACKNOWLEDGMENTS

We should like to thank the members of the Language Based Technology Section at DTU Informatics for their collaboration on Flow Logic; in particular, Jörg Kreiker, René Rydhof Hansen, Christoffer Rosenkilde Nielsen, Christian Probst, and Terkel Tolstrup. A special thanks goes to our students Piotr Filipiuk, Alejandro Hernandez, Lei Song, and Fuyuan Zhang for working with us on Appendix A and the corresponding proofs in Appendix B.

## REFERENCES

- ABADI, M. 1999a. A calculus for cryptographic protocols: The Spi calculus. *Inform. Computation* 148, 1, 1–70.
- ABADI, M. 1999b. Secrecy by typing in security protocols. *J. ACM* 46, 5, 749–786.
- APT, K., BLAIR, H., AND WALKER, A. 1988. A theory of declarative programming. In *Foundations of Deductive Databases and Logic Programming*. Morgan-Kaufman, 89–148.
- APT, K. R. 1981. Ten years of Hoare’s logic: A survey - part 1. *ACM Trans. Program. Lang. Syst.* 3, 4, 431–483.
- BAUER, J., NIELSON, F., NIELSON, H. R., AND PILEGAARD, H. 2008. Relational analysis of correlation. In *Proceedings of the International Static Analysis Symposium (SAS)*. Lecture Notes in Computer Science, vol. 5079, Springer, 32–46.
- BETTINI, L., BONO, V., DE NICOLA, R., FERRARI, G., GORLA, D., LORETI, M., MOGGI, E., PUGLIESE, R., TUOSTO, E., AND VENNARI, B. 2003. The Klaim project: Theory and practice. In *Global Computing - Programming Environments, Languages, Security and Analysis of Systems*. Lecture Notes in Computer Science, vol. 2874, Springer-Verlag.
- BODEI, C., DEGANI, P., NIELSON, F., AND NIELSON, H. R. 1998. Control flow analysis for the  $\pi$ -calculus. In *Proceedings of the International Conference on Concurrency Theory (CONCUR)*. Lecture Notes in Computer Science, vol. 1466, Springer, 84–98.
- BODEI, C., DEGANI, P., NIELSON, F., AND NIELSON, H. R. 1999. Static analysis of processes for no read-up and no write-down. In *Proceedings of the International Conference on Foundations of Software Science and Computation Structures (FOSSACS)*. Lecture Notes in Computer Science, vol. 1578, Springer, 120–134.
- BODEI, C., DEGANI, P., NIELSON, F., AND NIELSON, H. R. 2001a. Static analysis for the  $\pi$ -calculus with applications to security. *Inform. Comput.* 168, 1, 68–92.
- BODEI, C., DEGANI, P., NIELSON, H. R., AND NIELSON, F. 2001b. Static analysis for secrecy and non-interference in networks of processes. In *Proceedings of the International Conference on Parallel Computing Technologies (PACT)*. Lecture Notes in Computer Science, vol. 2127, Springer, 27–41.
- BODEI, C., DEGANI, P., NIELSON, H. R., AND NIELSON, F. 2002. Flow logic for Dolev-Yao secrecy in cryptographic processes. *Future Gen. Comput. Syst.* 18, 6, 747–756.
- BODEI, C., BUCHHOLTZ, M., DEGANI, P., NIELSON, F., AND NIELSON, H. R. 2003. Automatic validation of protocol narration. In *Proceedings of IEEE Computer Security Foundations Workshop (CSFW)*. IEEE Press, 126–140.
- BODEI, C., BUCHHOLTZ, M., DEGANI, P., NIELSON, F., AND NIELSON, H. R. 2005. Static validation of security protocols. *J. Comput. Security* 13, 3, 347–390.
- BODEI, C., DEGANI, P., AND PRIAMI, C. 2005. Checking security policies through an enhanced control flow analysis. *J. Comput. Security* 13, 1, 49–85.
- BODEI, C., DEGANI, P., GAO, H., AND BRODO, L. 2007. Detecting and preventing type flaws: A control flow analysis with tags. *Electr. Notes Theor. Comput. Sci.* 194, 1, 3–22.



- BOLANDER, T. AND HANSEN, R. R. 2007. Hybrid logical analyses of the ambient calculus. In *Proceedings of the Workshop on Logic, Language, Information and Computation (WoLLIC)*. Lecture Notes in Computer Science, vol. 4576, Springer, 83–100.
- BUGLIESI, M., CASTAGNA, G., AND CRAFA, S. 2001. Boxed Ambients. In *Proceedings of the International Symposium on Theoretical Aspects of Computer Software (TACS)*. Lecture Notes in Computer Science, vol. 2215, 38–63.
- CARDELLI, L. AND GORDON, A. D. 2000. Mobile Ambients. *Theor. Comput. Sci.* 240, 1, 177–213.
- CHANDRA, A. AND HAREL, D. 1980. Computable queries for relational data bases. *J. Comput. System Sci.* 21, 2, 156–178.
- DE NICOLA, R., FERRARI, G. L., AND PUGLIESE, R. 1998. Klaim: A kernel language for agents interaction and mobility. *IEEE Trans. Softw. Eng.* 24, 5, 315–330.
- DE NICOLA, R., GORLA, D., HANSEN, R. R., NIELSON, F., NIELSON, H. R., PROBST, C. W., AND PUGLIESE, R. 2008. From flow logic to static type systems for coordination languages. In *Proceedings of the International Conference on Coordination Models and Languages (Coordination)*. Lecture Notes in Computer Science, vol. 5052, Springer, 100–116.
- DE NICOLA, R., GORLA, D., HANSEN, R. R., NIELSON, F., NIELSON, H. R., PROBST, C. W., AND PUGLIESE, R. 2010. From Flow Logic to static type systems for coordination languages. *Sci. Comput. Progr.*
- FERET, J. 2002. Dependency analysis of mobile systems. In *Proceedings of the European Symposium on Programming (ESOP)*. Lecture Notes in Computer Science, vol. 2305, 314–330.
- GORLA, D. AND PUGLIESE, R. 2003. Resource access and mobility control with dynamic privileges acquisition. In *Proceedings of the International Colloquium on Automata, Languages and Programming (ICALP)*. Lecture Notes in Computer Science, vol. 2719, Springer, 119–132.
- HANSEN, R. R., JENSEN, J. G., NIELSON, F., AND NIELSON, H. R. 1999. Abstract interpretation of Mobile Ambients. In *Proceedings of International Static Analysis Symposium (SAS)*. Lecture Notes in Computer Science, vol. 1694, Springer, 134–148.
- HANSEN, R. R., PROBST, C. W., AND NIELSON, F. 2006. Sandboxing in myKlaim. In *Proceedings of the 1st International Conference on Availability, Reliability and Security (ARES)*. IEEE Computer Society, 174–181.
- HANSEN, R. R., NIELSON, F., NIELSON, H. R., AND PROBST, C. W. 2008. Static validation of licence conformance policies. In *Proceedings of the 3rd International Conference on Availability, Reliability and Security (ARES)*. IEEE Computer Society, 1104–1111.
- HENNESSY, M. 2007. *A Distributed Pi-Calculus*. Cambridge University Press, Cambridge, UK.
- LEVI, F. AND SANGIORGI, D. 2003. Mobile safe ambients. *ACM Trans. Program. Lang. Syst.* 25, 1, 1–69.
- MCALLESTER, D. A. 1999. On the complexity analysis of static analyses. In *Proceedings of the International Static Analysis Symposium (SAS)*. Lecture Notes in Computer Science, vol. 1694, Springer, 312–329.
- MILNER, R. 1999. *Communicating and Mobile Systems: the Pi-Calculus*. Cambridge University Press, Cambridge, UK.
- NIELSON, F. AND NIELSON, H. R. 2007a. Heuristics for safety and security constraints. *Electronic Notes in Theoretical Computer Science* 172, 523–543.
- NIELSON, F. AND NIELSON, H. R. 2010. Model checking is static analysis of modal logic. In *Proceedings of the International Conference on Foundations of Software Science and Computation Structures (FoSSaCS)*. Lecture Notes in Computer Science, Springer.
- NIELSON, F., NIELSON, H. R., AND HANKIN, C. L. 1999. *Principles of Program Analysis*. Springer.
- NIELSON, F., NIELSON, H. R., AND HANSEN, R. R. 2002. Validating firewalls using flow logics. *Theoret. Comput. Sci.* 283, 2, 381–418.
- NIELSON, F., NIELSON, H. R., AND SEIDL, H. 2002a. Cryptographic analysis in cubic time. *Electron. Notes Theoret. Comput. Sci.* 62, 7–23.
- NIELSON, F., NIELSON, H. R., AND SEIDL, H. 2002b. A Succinct solver for ALFP. *Nordic J. Comput.* 9, 335–372.
- NIELSON, F., NIELSON, H. R., PRIAMI, C., AND ROSA, D. 2007. Control flow analysis for BioAmbients. *Electron. Notes Theoret. Comput. Sci.* 180, 3, 65–79.
- NIELSON, F., NIELSON, H. R., BAUER, J., NIELSEN, C. R., AND PILEGAARD, H. 2008. Relational analysis for delivery of services. In *Proceedings of the Trustworthy Global Computing (TGC) Conference*. Lecture Notes in Computer Science, vol. 4912, Springer, 73–89.
- NIELSON, H. R. AND NIELSON, F. 2002. Flow Logic: A multi-paradigmatic approach to static analysis. In *The Essence of Computation: Complexity, Analysis, Transformation, Essays dedicated to Neil D. Jones*. Lecture Notes in Computer Science, vol. 2566, Springer, 223–244.



- NIELSON, H. R. AND NIELSON, F. 2007b. A flow-sensitive analysis of privacy properties. In *Proceedings of the Computer Security Foundations Symposium (CFS)*. IEEE Computer Society, 249–264.
- NIELSON, H. R. AND NIELSON, F. 2009. A monotone framework for CCS. *Comput. Lang. Syst. Struct.* 35, 4, 365–394.
- NIELSON, H. R., NIELSON, F., AND PILEGAARD, H. 2004. Spatial analysis of BioAmbients. In *Proceedings of the International Static Analysis Symposium (SAS)*. Lecture Notes in Computer Science. Springer, 69–83.
- NIELSON, H. R., NIELSON, F., AND BUCHHOLTZ, M. 2005. Security for mobility. In *Foundations of Security Analysis and Design II*, R. Focardi and R. Gorrieri Eds., Lecture Notes in Computer Science, vol. 2946, Springer, 207–266.
- PARROW, J. 2001. An introduction to the  $\pi$ -calculus. In *Handbook of Process Algebra*, J. A. Bergstra, A. Ponse, and S. A. Smolka Eds., Elsevier, 479–544.
- PILEGAARD, H., NIELSON, F., AND NIELSON, H. R. 2006a. Active evaluation contexts for reaction semantics. In *Proceedings of the Workshop on Structural Operational Semantics (SOS)*. Electronic Notes in Theoretical Computer Science, vol. 175, Elsevier, 57–70.
- PILEGAARD, H., NIELSON, F., AND NIELSON, H. R. 2006b. Context dependent analysis of BioAmbients. In *Proceedings of the 1st International Workshop on Emerging Applications of Abstract Interpretation (EAAI)*.
- PILEGAARD, H., NIELSON, F., AND NIELSON, H. R. 2008. Pathway analysis for BioAmbients. *J. Logic Algebr. Prog.* 77, 92–130.
- PROBST, C. W., HANSEN, R. R., AND NIELSON, F. 2007. Where can an insider attack? In *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*. Lecture Notes in Computer Science, vol. 4691, Springer, 127–142.
- REGEV, A., PANINA, E. M., SILVERMAN, W., CARDELLI, L., AND SHAPIRO, E. Y. 2004. BioAmbients: An abstraction for biological compartments. *Theor. Comput. Sci.* 325, 1, 141–167.
- SHIVERS, O. 1988. Control flow analysis in Scheme. In *Proceedings of the Conference on Programming Language Design and Implementation (PLDI)*. ACM, New York, NY, 164–174.
- TARSKI, A. 1955. A lattice-theoretic fixpoint theorem and its applications. *Pacific J. Math.* 5, 2, 285–309.
- TOLSTRUP, T. K., NIELSON, F., AND HANSEN, R. R. 2007. Locality-based security policies. In *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*. Lecture Notes in Computer Science, vol. 4691, Springer, 185–201.
- VENET, A. 1998. Automatic determination of communication topologies in mobile systems. In *Proceedings of the International Static Analysis Symposium (SAS)*. Lecture Notes in Computer Science, vol. 1503, 152–167.

Received November 2009; revised April 2010; accepted April 2010